

COL1000: Introduction to Programming

Lectures 17-19

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Aliasing

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

```
A = [1, 2, 3]
```

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

```
A = [1, 2, 3]
B = A          # id(A) = id(B)
```


Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

```
A = [1, 2, 3]
B = A          # id(A) = id(B)
B[0] = 11
```


Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

```
A = [1, 2, 3]
B = A          # id(A) = id(B)
B[0] = 11
print(A)       # prints [11, 2, 3]
```

Aliasing

Aliasing in python occurs when **multiple** variables **refer the same object** in the memory.

Note: The **assignment operator** (=) b/w objects always **create an alias**.

Example:

```
A = [1, 2, 3]
B = A          # id(A) = id(B)
B[0] = 11
print(A)       # prints [11, 2, 3]
```

Aliasing in Integers

Aliasing in Integers

Program

```
x = 2233  
print(x)  
print("address x:", id(x))
```

Aliasing in Integers

Program

```
x = 2233  
print(x)  
print("address x:", id(x))
```

Memory:



x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))
```

Memory:



x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))
```

Memory:



x
y

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2
```

Memory:



x
y

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2
```

Memory:



y

x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2

print(x)
print("address x:", id(x))
```

Memory:



y

x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2

print(x)
print("address x:", id(x))
```

Output

```
2233
address x: 112233445500022
```

Memory:



y

x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2

print(x)
print("address x:", id(x))
```

Output

```
2233
address x: 112233445500022

2233
address y: 112233445500022
```

Memory:



y

x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2

print(x)
print("address x:", id(x))
```

Output

```
2233
address x: 112233445500022

2233
address y: 112233445500022

4466
```

Memory:



y

x

Aliasing in Integers

Program

```
x = 2233
print(x)
print("address x:", id(x))

y = x
print(y)
print("address y:", id(y))

x = x * 2

print(x)
print("address x:", id(x))
```

Output

```
2233
address x: 112233445500022

2233
address y: 112233445500022

4466
address x: 112233009940940
```

Memory:



y

x

Aliasing in Function calls

Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```


Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:



Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:



Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:

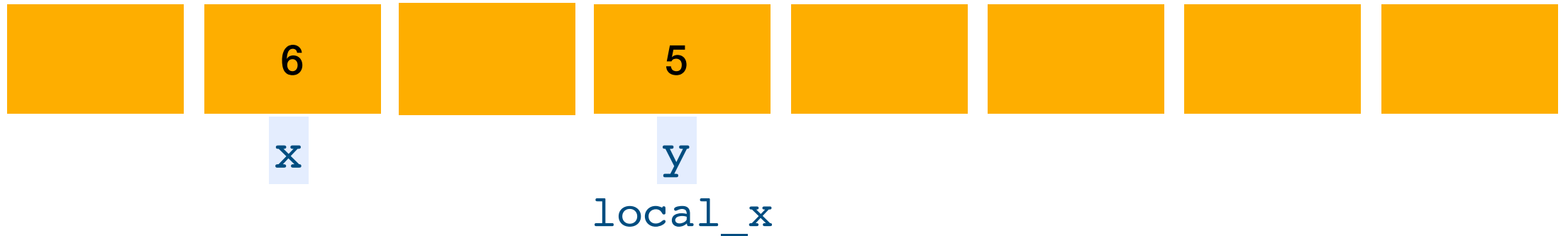


Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:



Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:

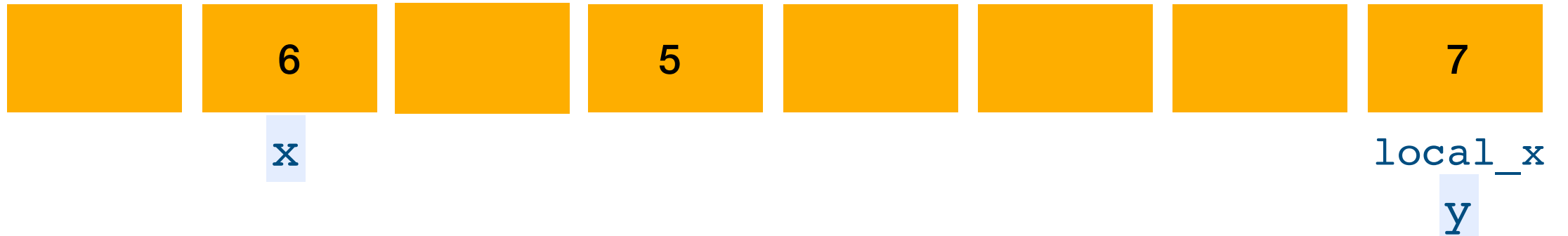


Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Memory:



Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    print(x, y)  
    return x  
  
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Output

```
7 5  
6 7
```

Memory:



Aliasing in Function calls

Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    y = x  
    print(x, y)  
    return x
```

```
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    y = x  
    print(x, y)  
    return x
```

```
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

Output

```
Error: cannot access local  
variable 'y' where it is not  
associated with a value.
```

Aliasing in Function calls

Program

```
def update(x):  
    x = y + 2  
    y = x  
    print(x, y)  
    return x
```

```
x = 6  
y = 5  
y = update(y)  
print(x, y)
```

treated as:

```
local_x = local_y + 2  
local_y = local_x
```

Output

```
Error: cannot access local  
variable 'y' where it is not  
associated with a value.
```

How are lists implemented in Python?

How are lists implemented in Python?

```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

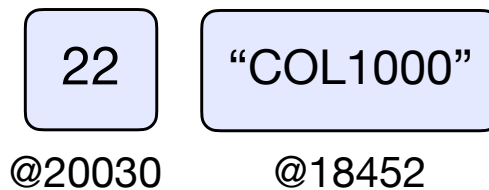
How are lists implemented in Python?

22

@20030

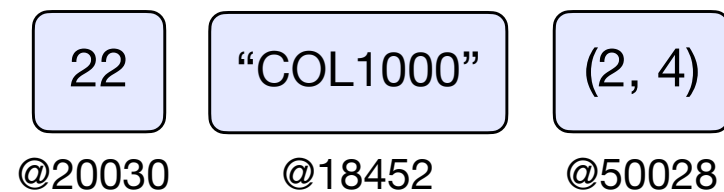
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



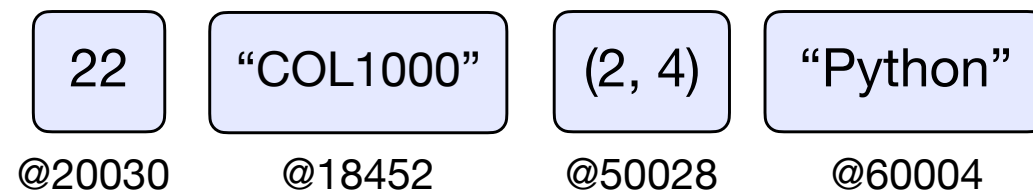
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```


How are lists implemented in Python?



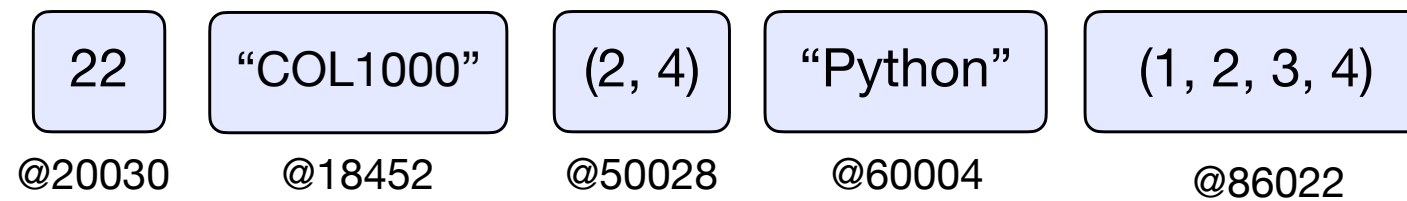
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```


How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



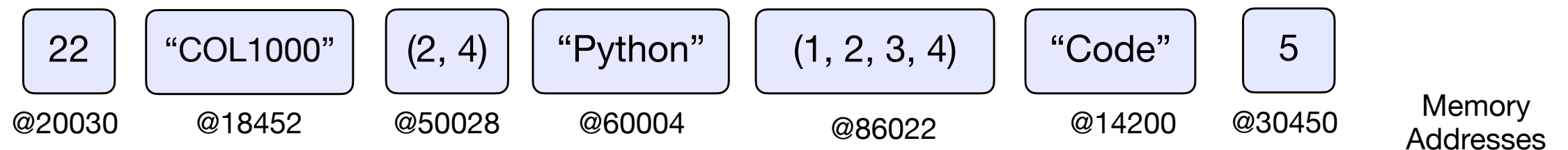
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?

Indexes

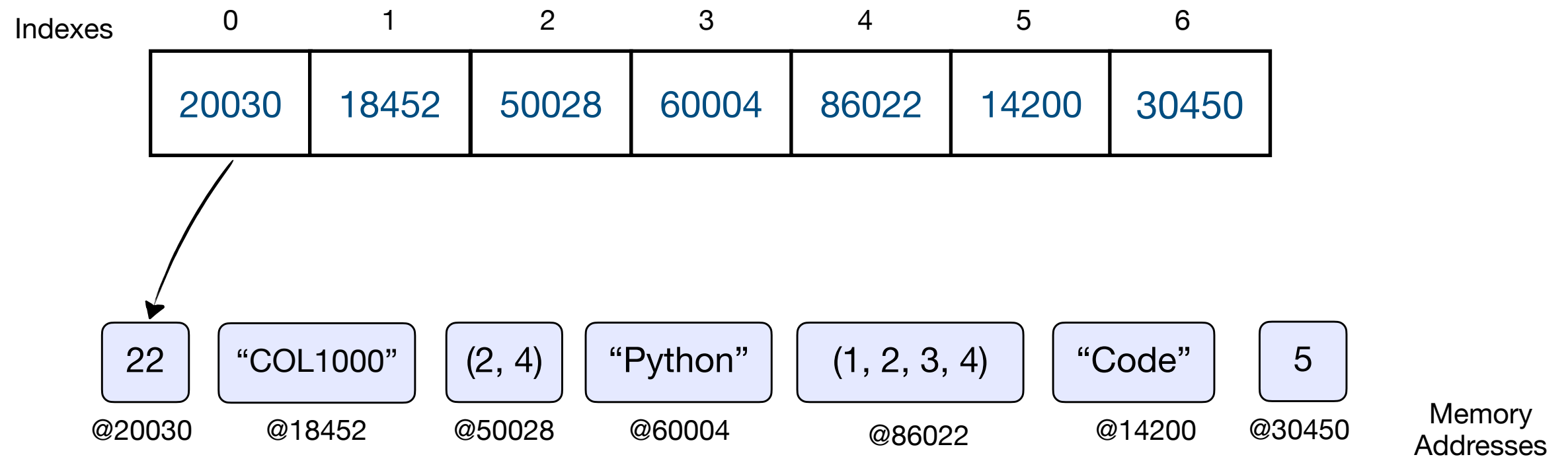
0	1	2	3	4	5	6
20030	18452	50028	60004	86022	14200	30450



Memory
Addresses

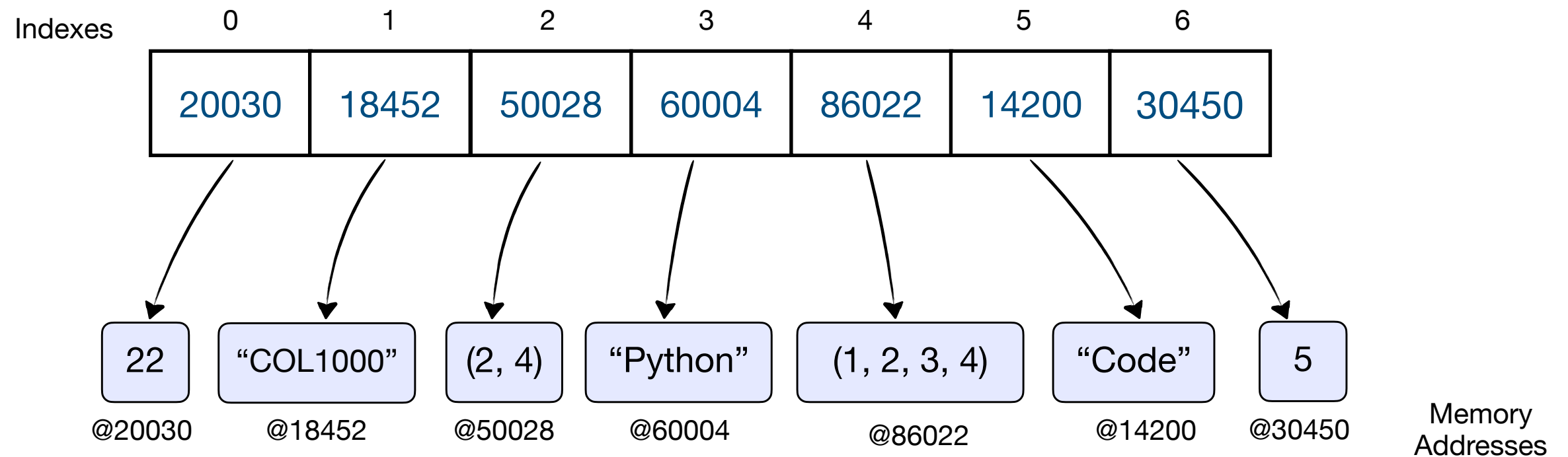
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

How are lists implemented in Python?



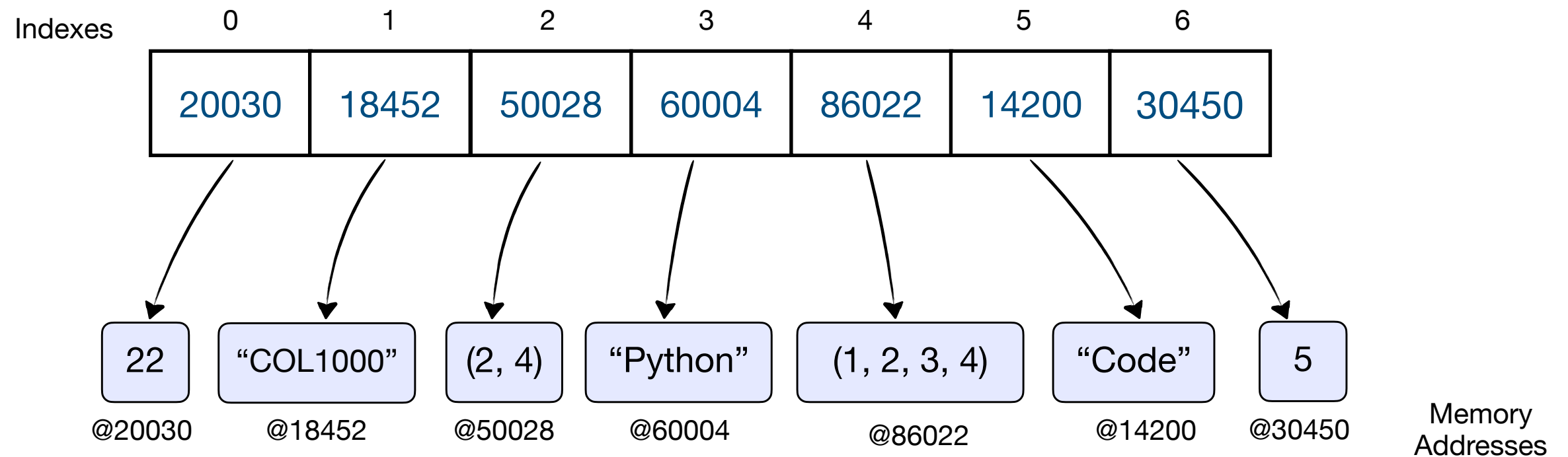
```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```


How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

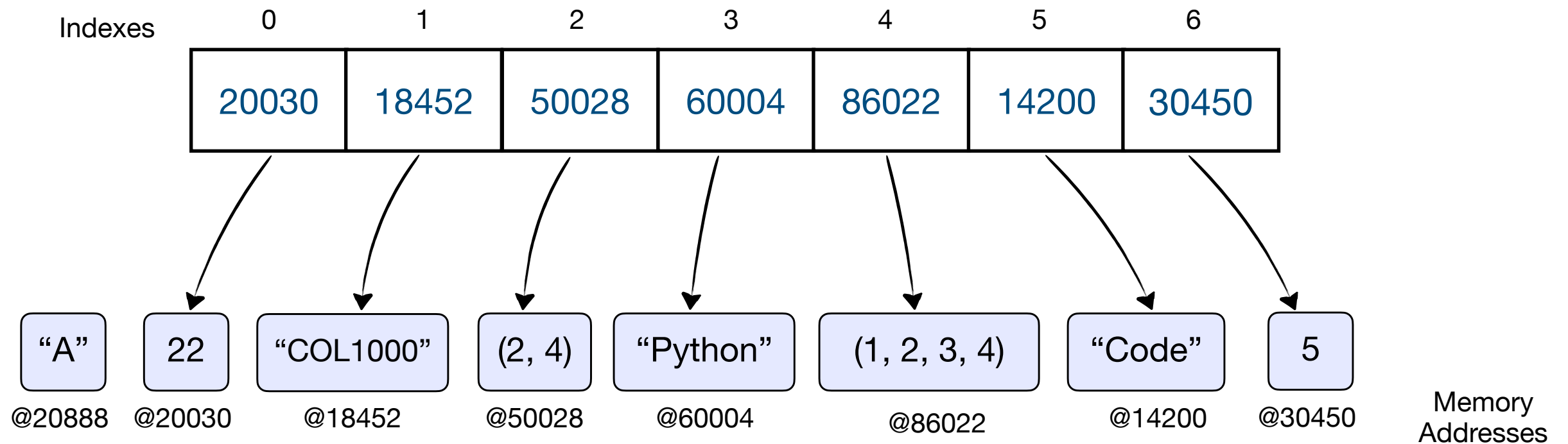

How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

```
L[0] = "A"
```

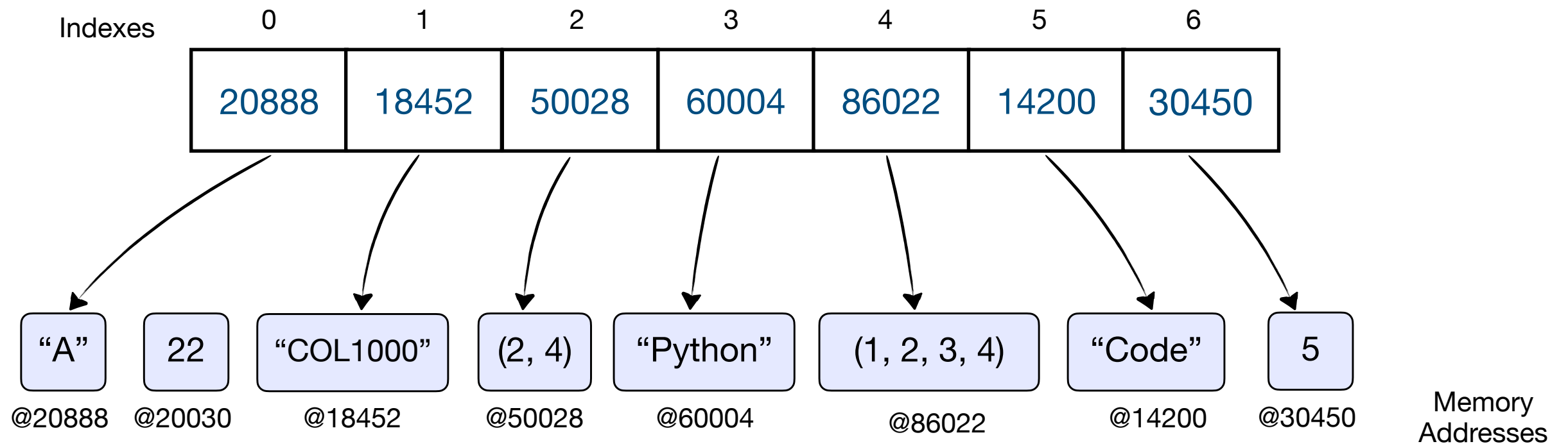
How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

```
L[0] = "A"
```

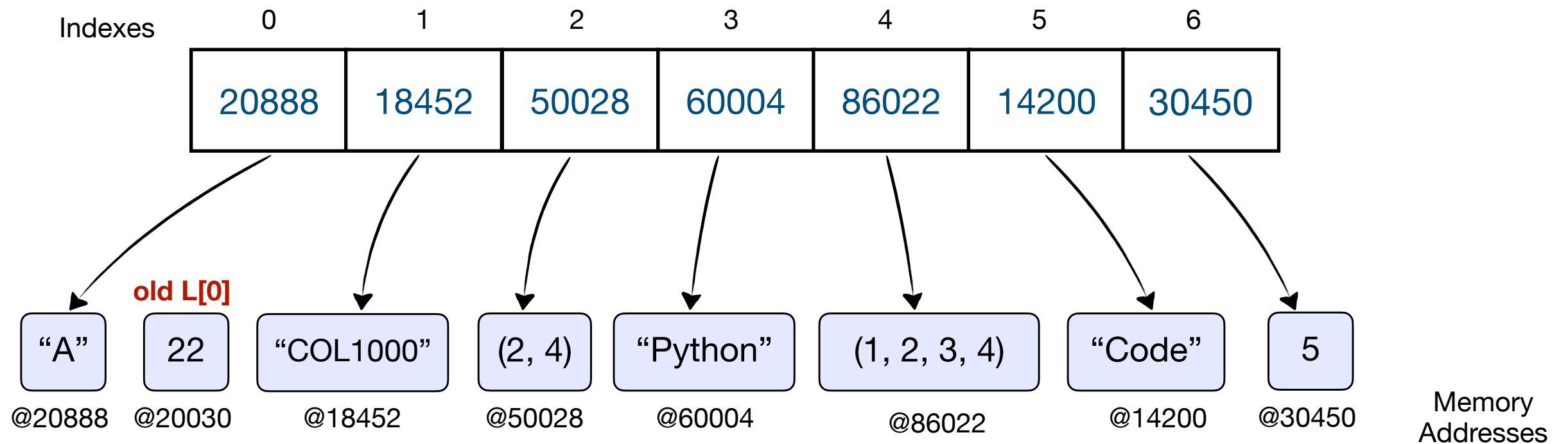
How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

```
L[0] = "A"
```

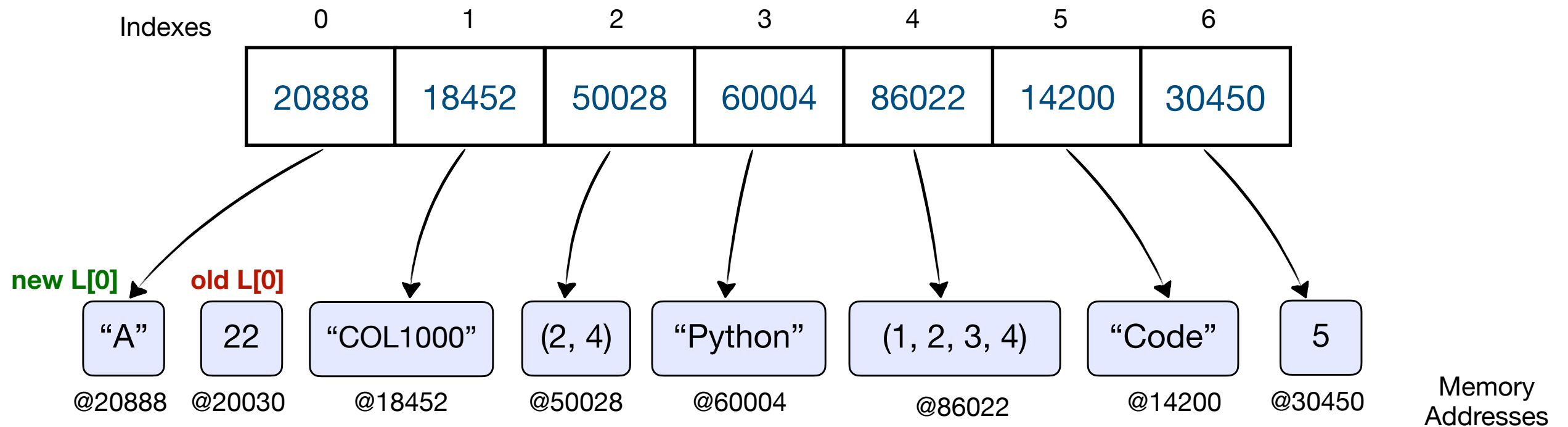
How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

```
L[0] = "A"
```

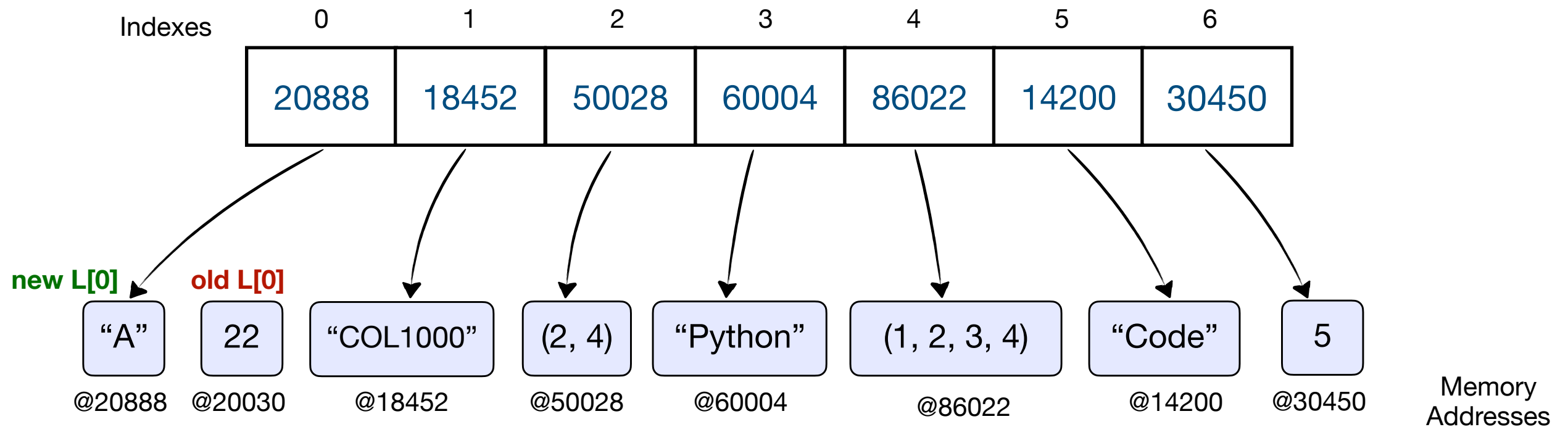
How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
```

```
L[0] = "A"
```

How are lists implemented in Python?



```
L = [22, "COL1000", (2,4), "Python", (1,2,3,4), "Code", 5]
L[0] = "A"
```

Thus, address of L[0] changes, but address of L remains same.

Aliasing in Lists

Aliasing in Lists

Program

```
A = [1, 2, 3]
```


Aliasing in Lists

Program

```
A = [1, 2, 3]
```

Aliasing in Lists

Program

```
A = [1, 2, 3]
```

1

@22004

2

@33002

Aliasing in Lists

Program

```
A = [1, 2, 3]
```

1

@22004

2

@33002

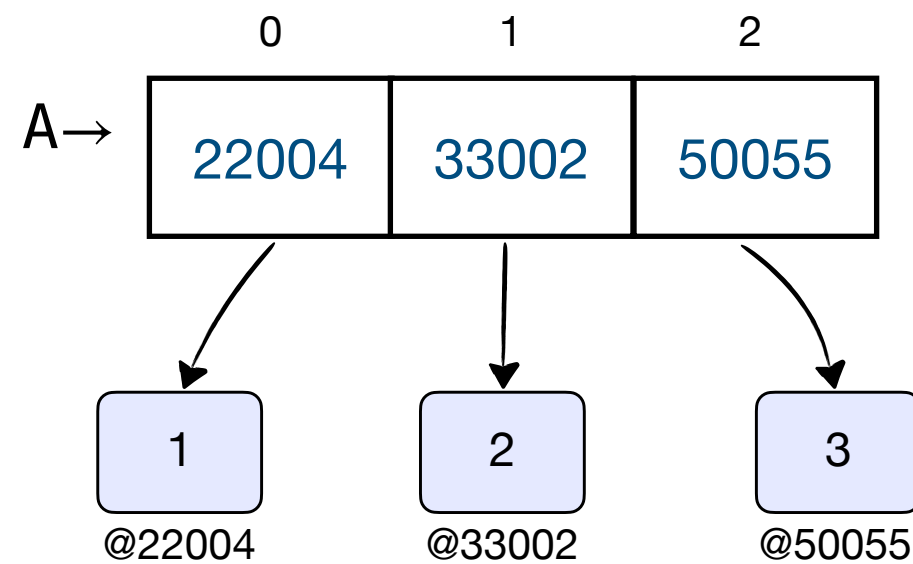
3

@50055

Aliasing in Lists

Program

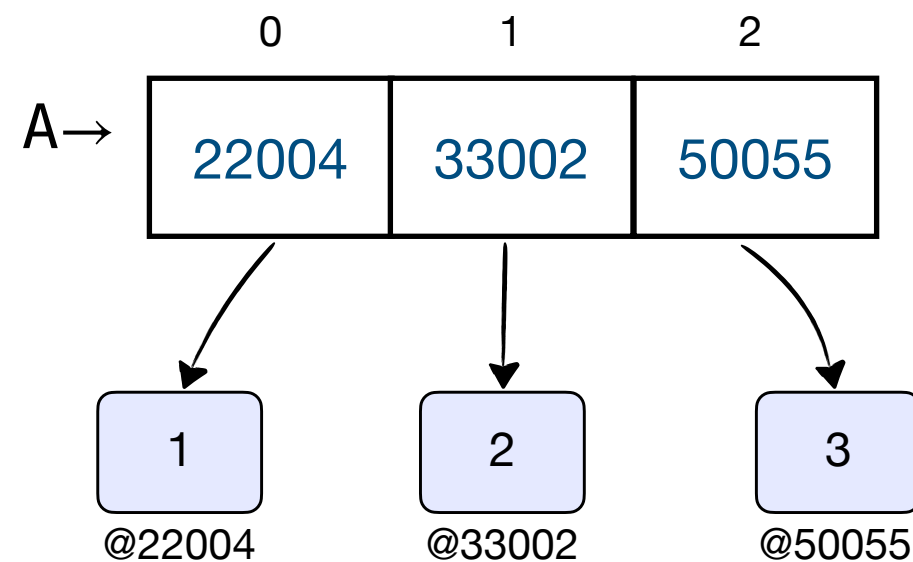
```
A = [1, 2, 3]
```



Aliasing in Lists

Program

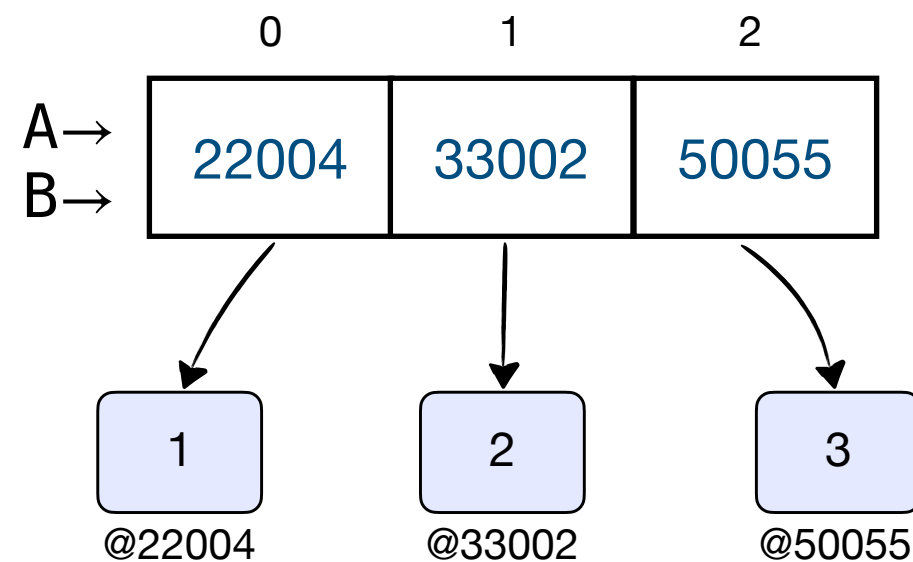
```
A = [1, 2, 3]  
B = A
```



Aliasing in Lists

Program

```
A = [1, 2, 3]  
B = A
```



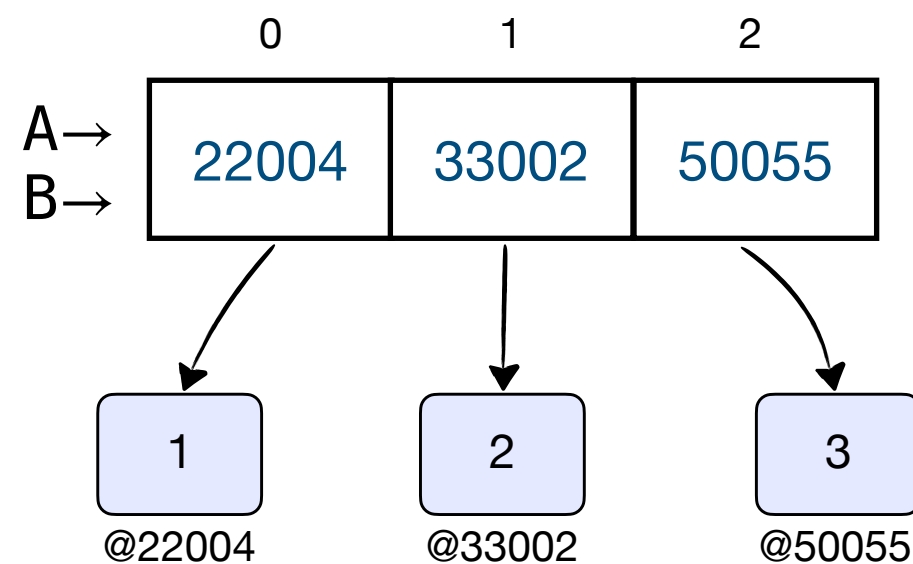
Aliasing in Lists

Program

```
A = [1, 2, 3]
```

```
B = A
```

```
B.append(11)
```



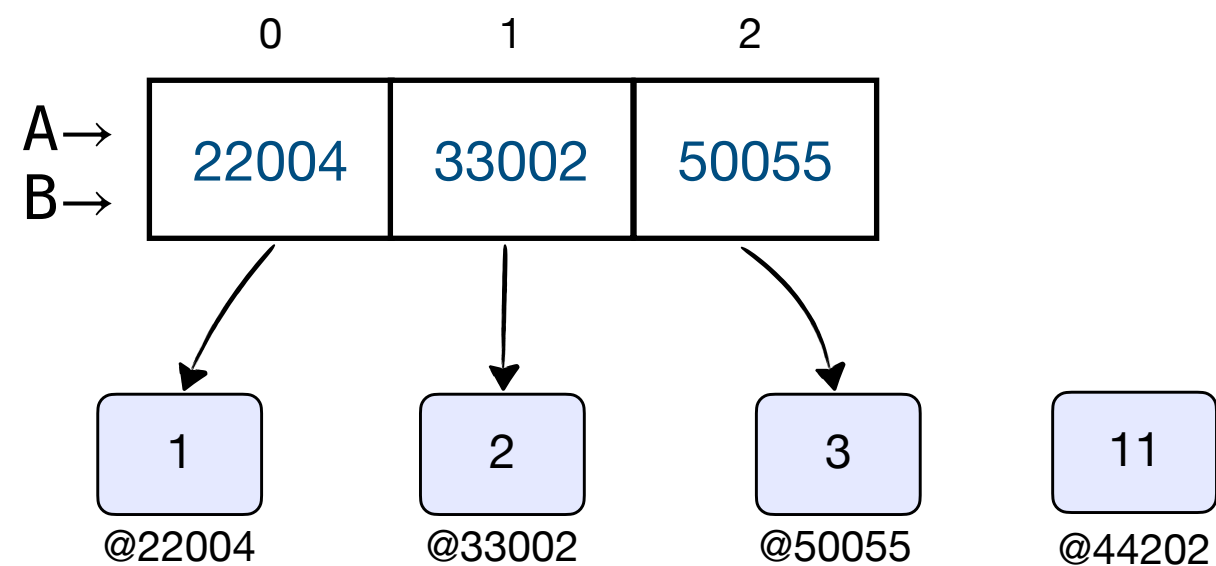
Aliasing in Lists

Program

```
A = [1, 2, 3]
```

```
B = A
```

```
B.append(11)
```



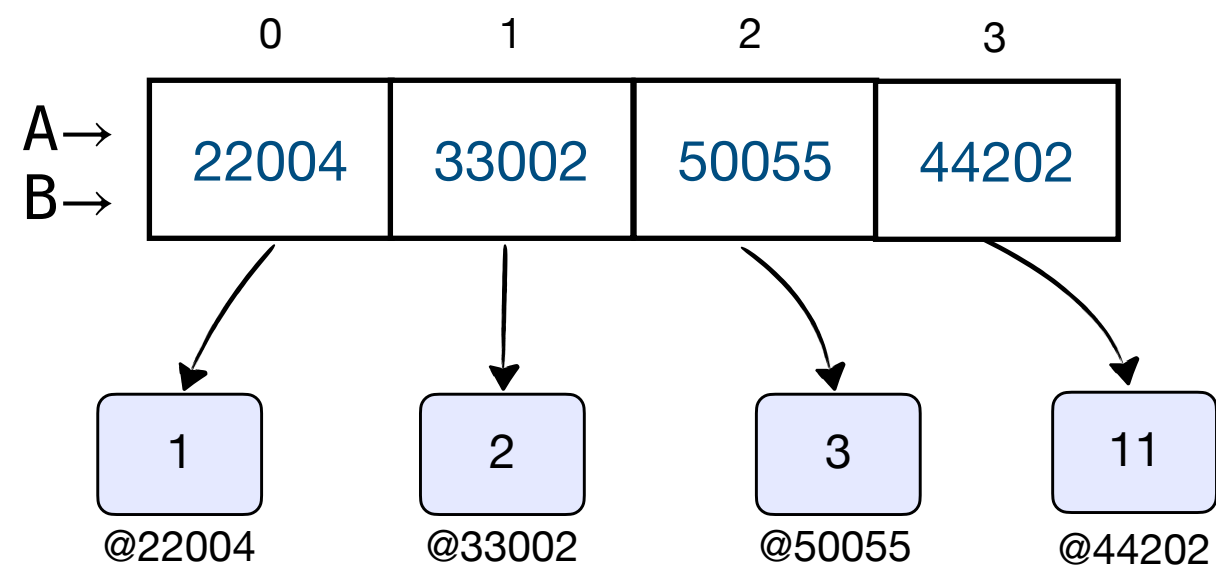
Aliasing in Lists

Program

```
A = [1, 2, 3]
```

```
B = A
```

```
B.append(11)
```



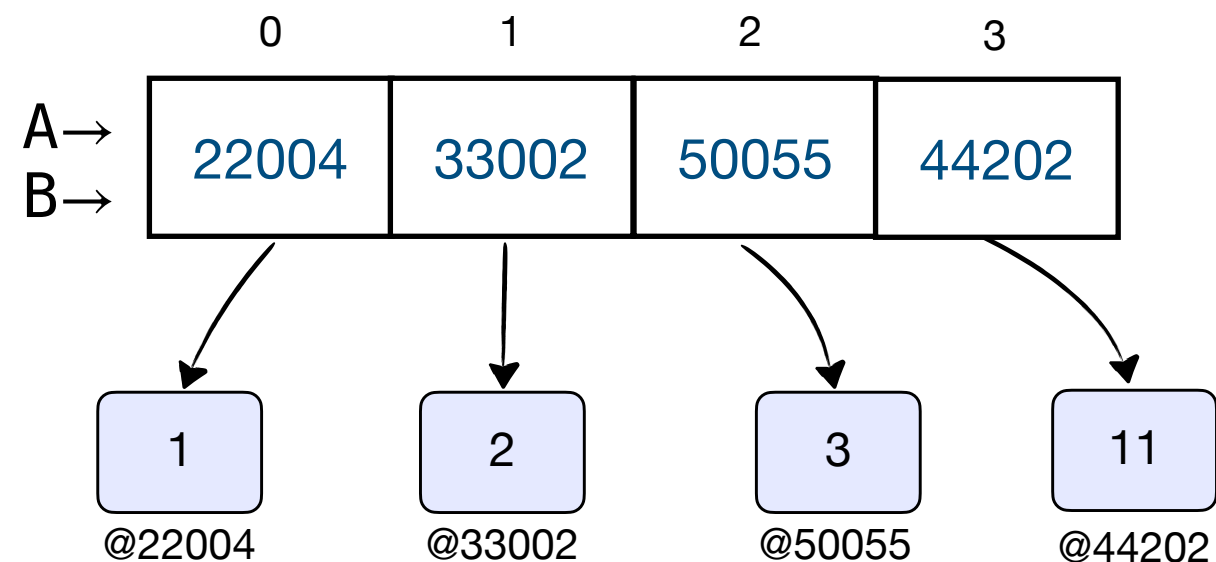
Aliasing in Lists

Program

```
A = [1, 2, 3]
B = A

B.append(11)

print(A)
print("address A:", id(A))
```



Aliasing in Lists

Program

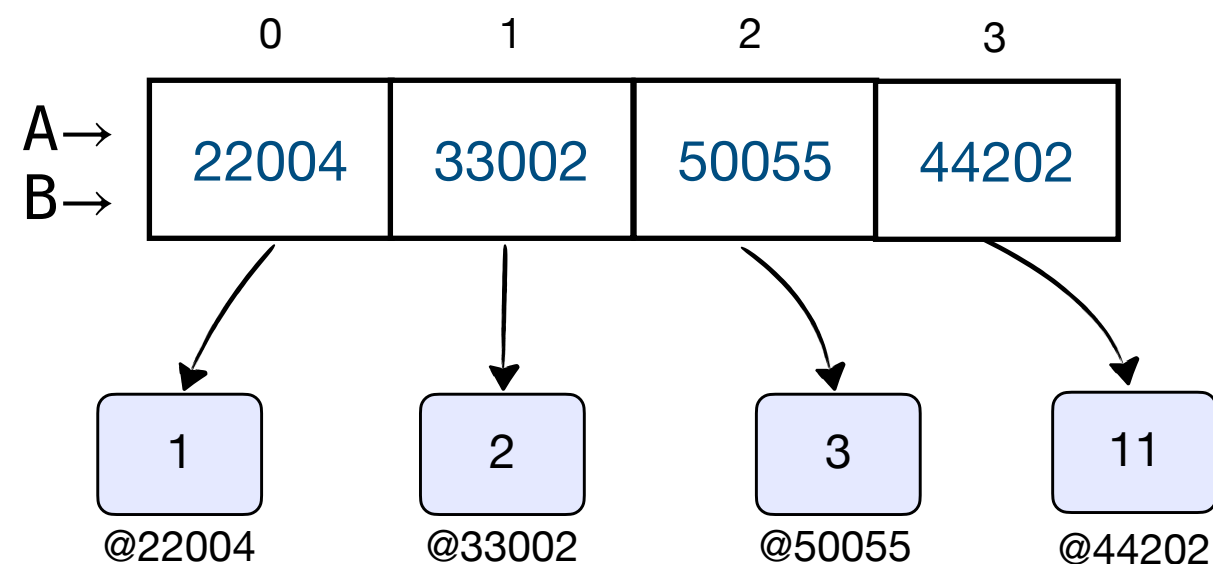
```
A = [1, 2, 3]
B = A

B.append(11)

print(A)
print("address A:", id(A))
```

Output

```
[1, 2, 3, 11]
address A: 112233445500022
```



Aliasing in Lists

Program

```
A = [1, 2, 3]
B = A

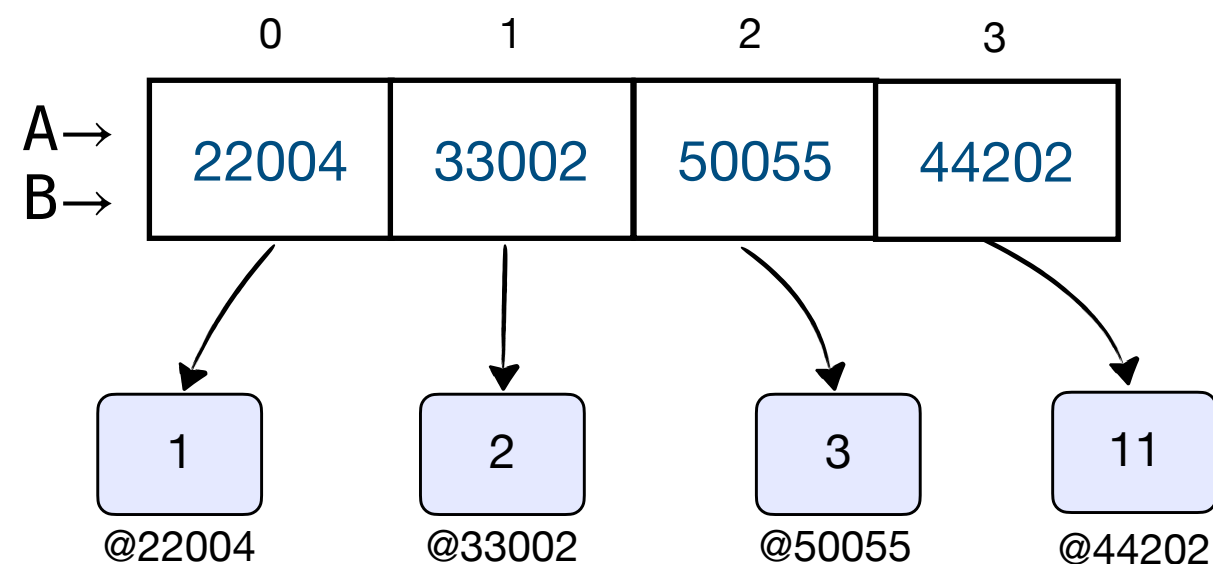
B.append(11)

print(A)
print("address A:", id(A))

print(B)
print("address B:", id(B))
```

Output

```
[1, 2, 3, 11]
address A: 112233445500022
```



Aliasing in Lists

Program

```
A = [1, 2, 3]
B = A

B.append(11)

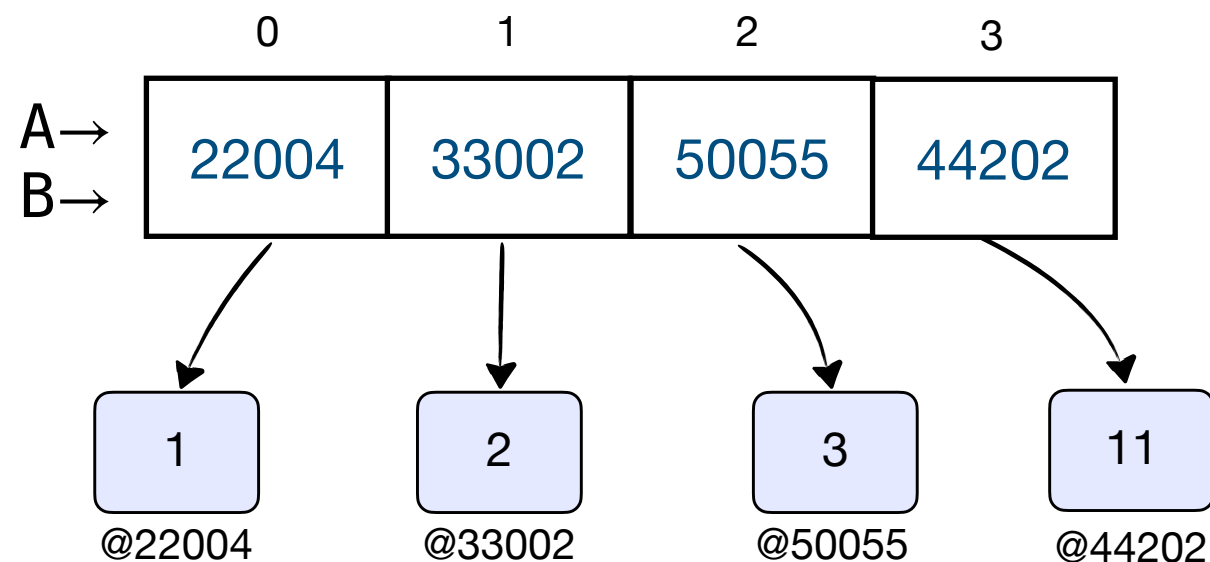
print(A)
print("address A:", id(A))

print(B)
print("address B:", id(B))
```

Output

```
[1, 2, 3, 11]
address A: 112233445500022

[1, 2, 3, 11]
address B: 112233445500022
```



Aliasing in Lists

Aliasing in Lists

Program

```
def f(M):  
    M[0] = 900  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```


Aliasing in Lists

Program

```
def f(M):  
    M[0] = 900  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Note: Inside the function, L and M reference to the same mutable list.

Aliasing in Lists

Program

```
def f(M):  
    M[0] = 900  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Output

Note: Inside the function, L and M reference to the same mutable list.

Aliasing in Lists

Program

```
def f(M):  
    M[0] = 900  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Output

```
[900, 502, 503]
```

Note: Inside the function, L and M reference to the same mutable list.

Aliasing in Lists

Program

```
def f(M):  
    M[0] = L[0] + M[0]  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Note: Inside the function, L and M reference to the same mutable list.

Aliasing in Lists

Program

```
def f(M):  
    M[0] = L[0] + M[0]  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Output

Note: Inside the function, L and M reference to the same mutable list.

Aliasing in Lists

Program

```
def f(M):  
    M[0] = L[0] + M[0]  
    return  
  
L = [501, 502, 503]  
f(L)  
print(L)
```

Output

```
[1002, 502, 503]
```

Note: Inside the function, L and M reference to the same mutable list.

What happens when we sort a list?

What happens when we sort a list?

```
L = [2222, 1111, 5555, 3333, 4444]
```

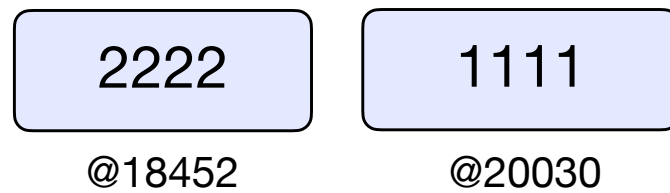

What happens when we sort a list?

2222

@18452

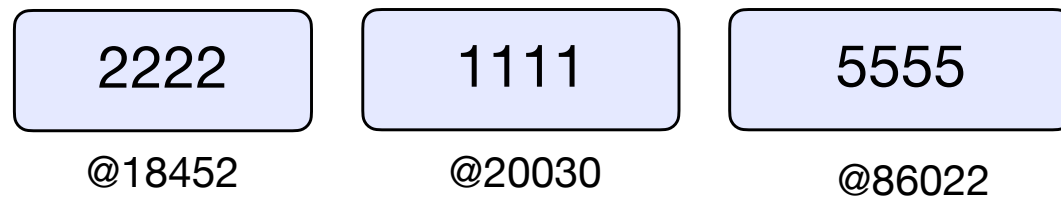
```
L = [2222, 1111, 5555, 3333, 4444]
```


What happens when we sort a list?



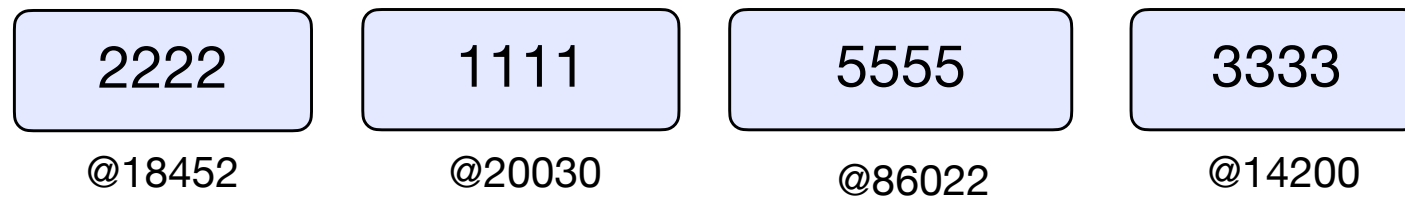
```
L = [2222, 1111, 5555, 3333, 4444]
```

What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

What happens when we sort a list?



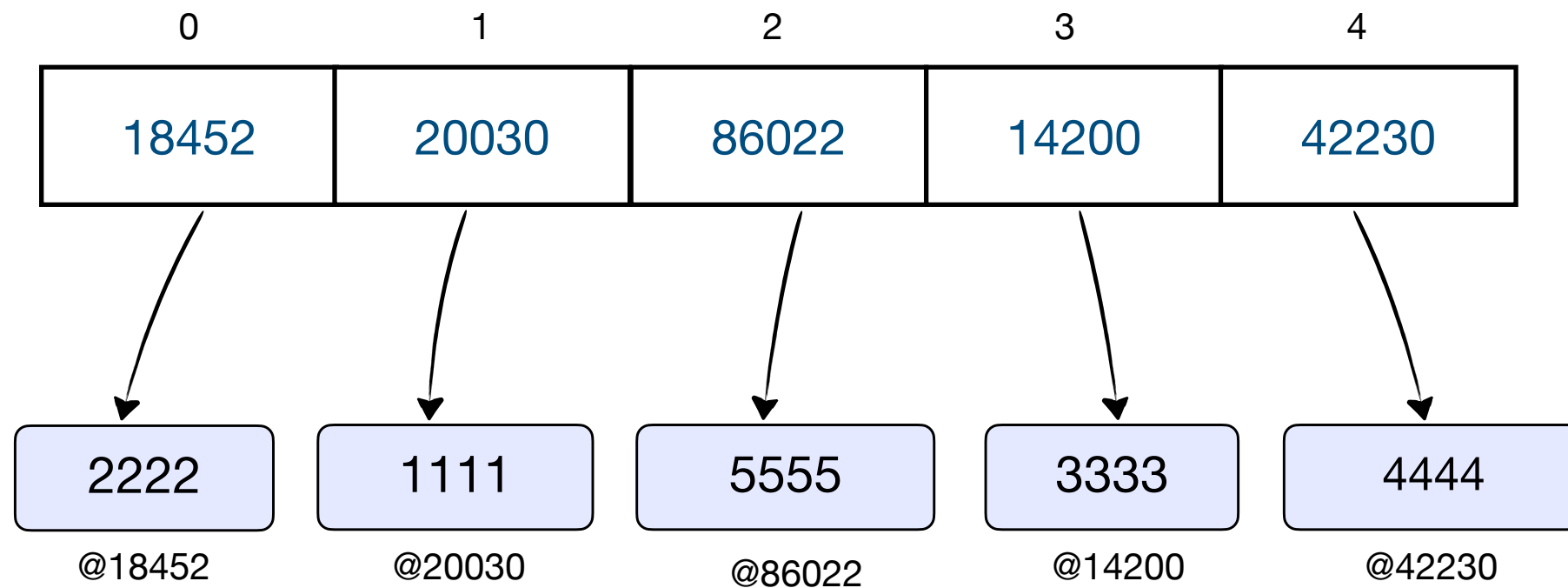
```
L = [2222, 1111, 5555, 3333, 4444]
```

What happens when we sort a list?



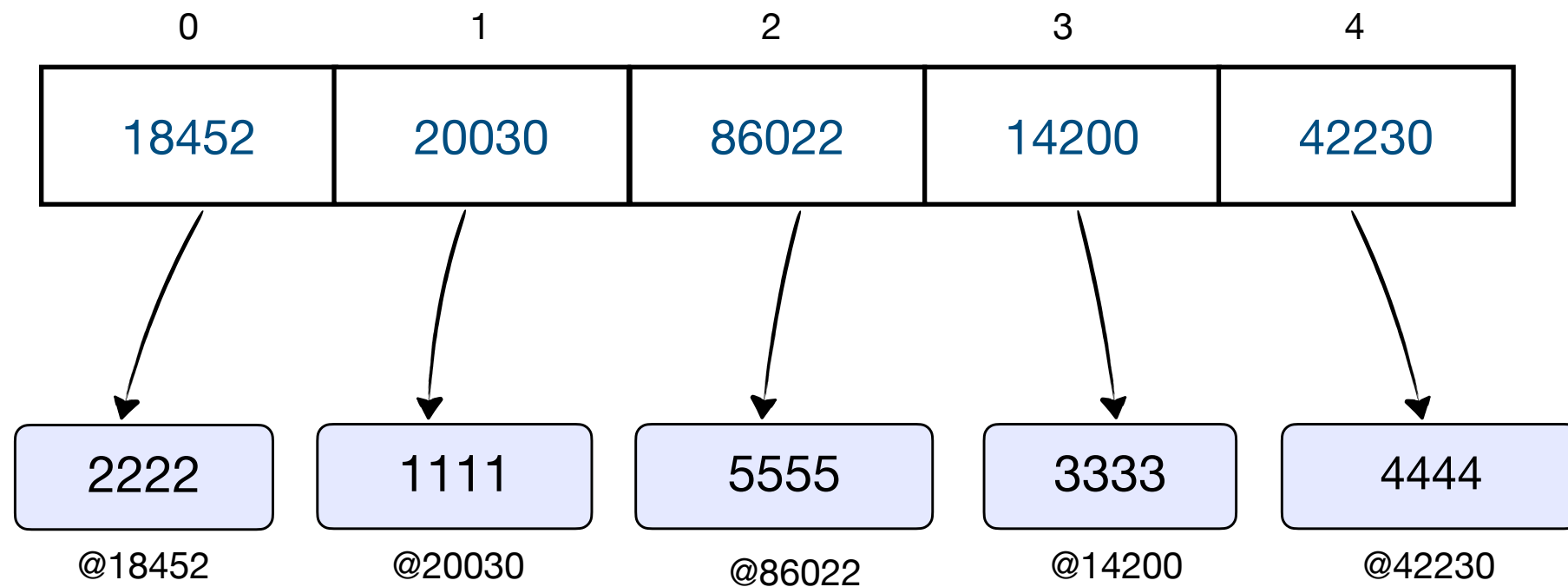
```
L = [2222, 1111, 5555, 3333, 4444]
```

What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

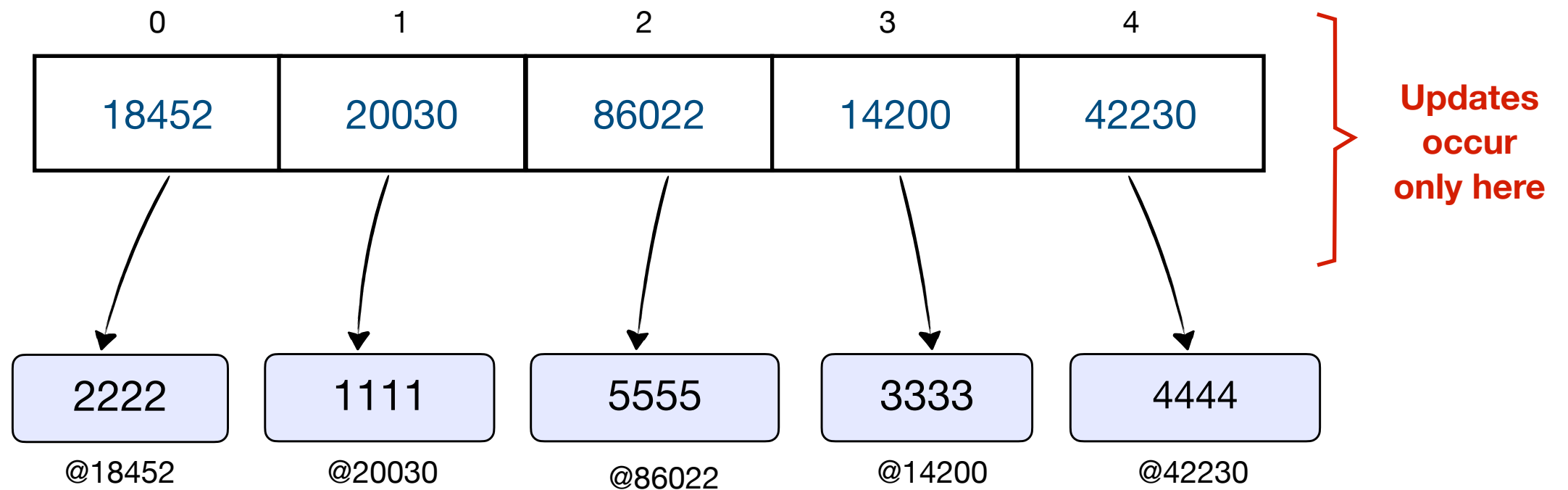
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

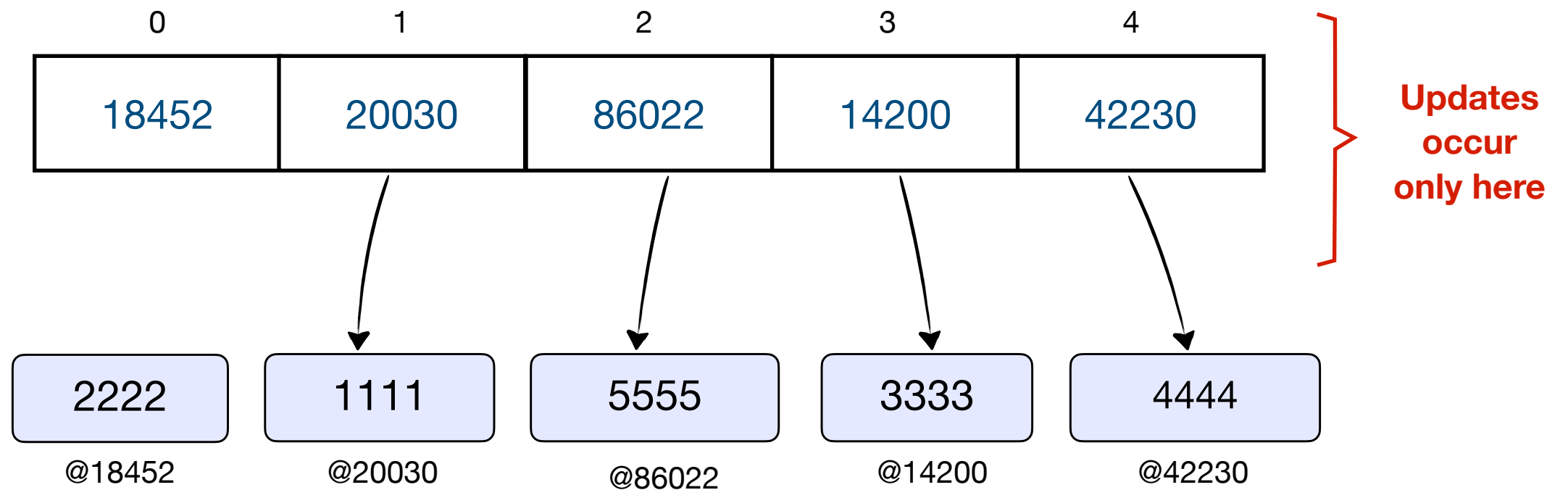
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

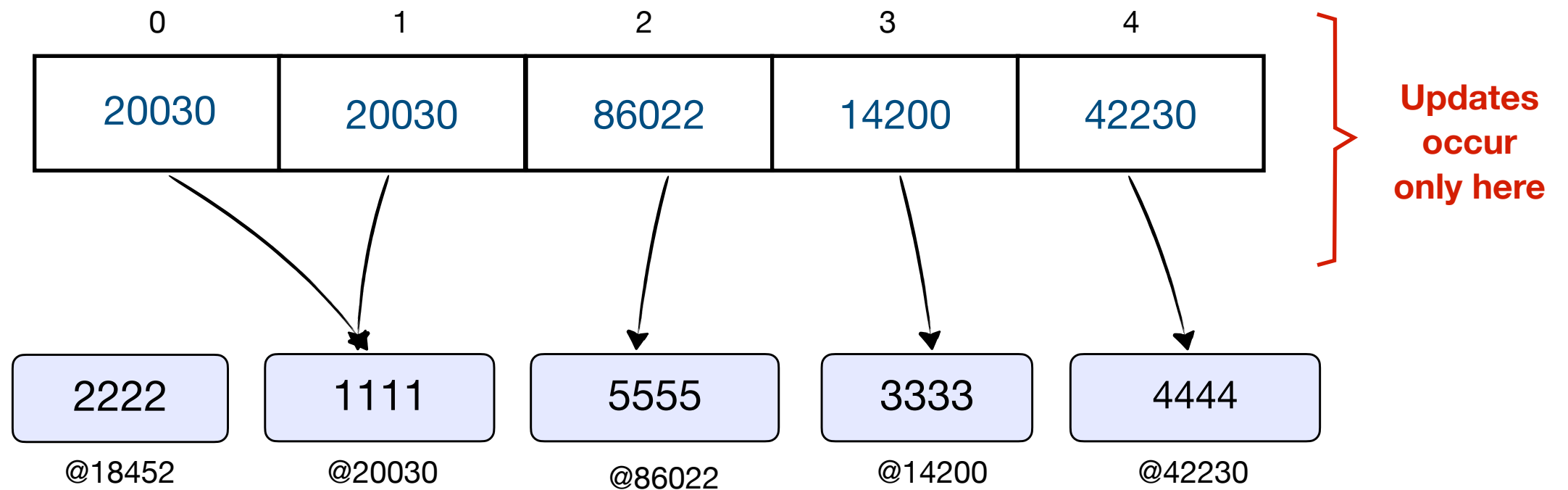

What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

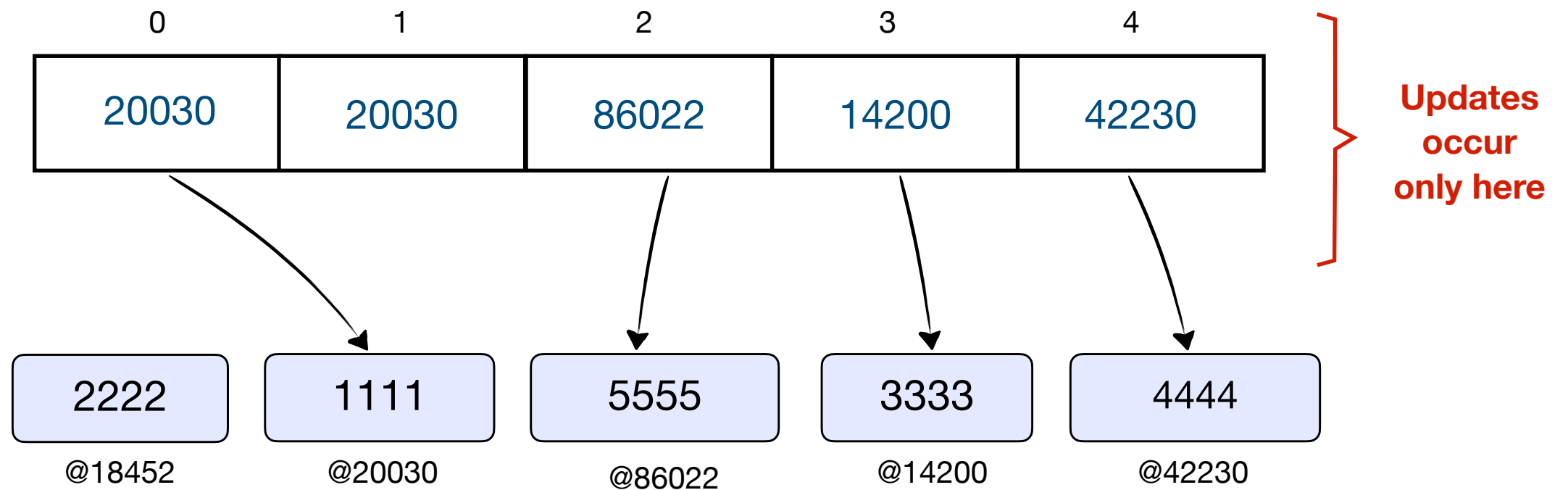

What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

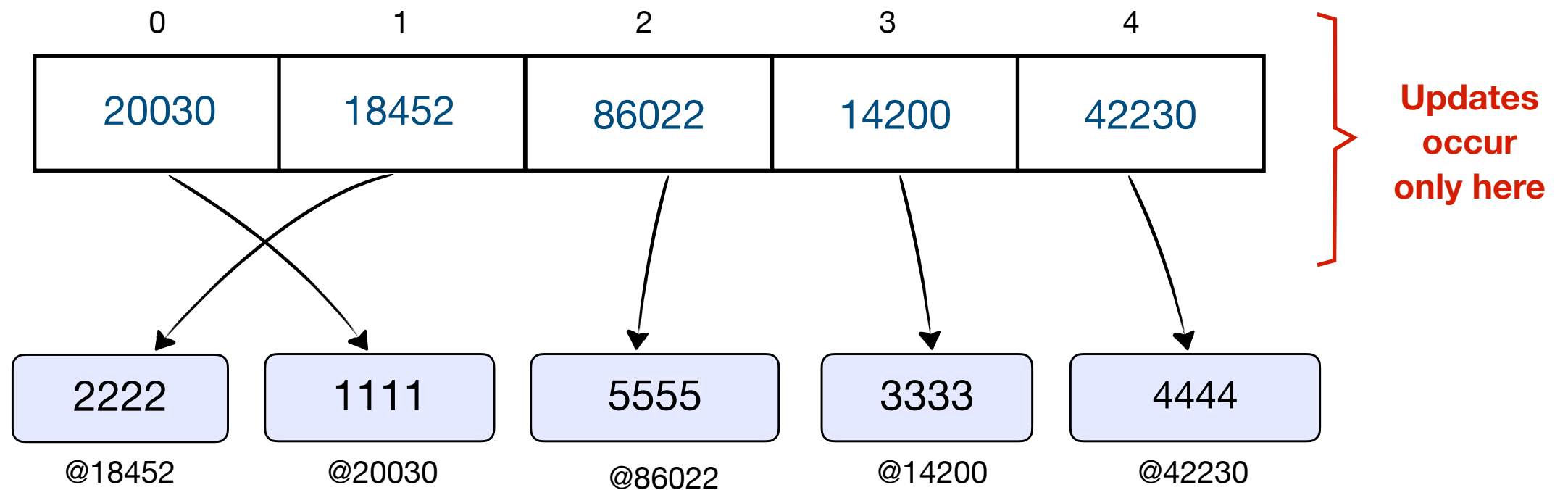
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

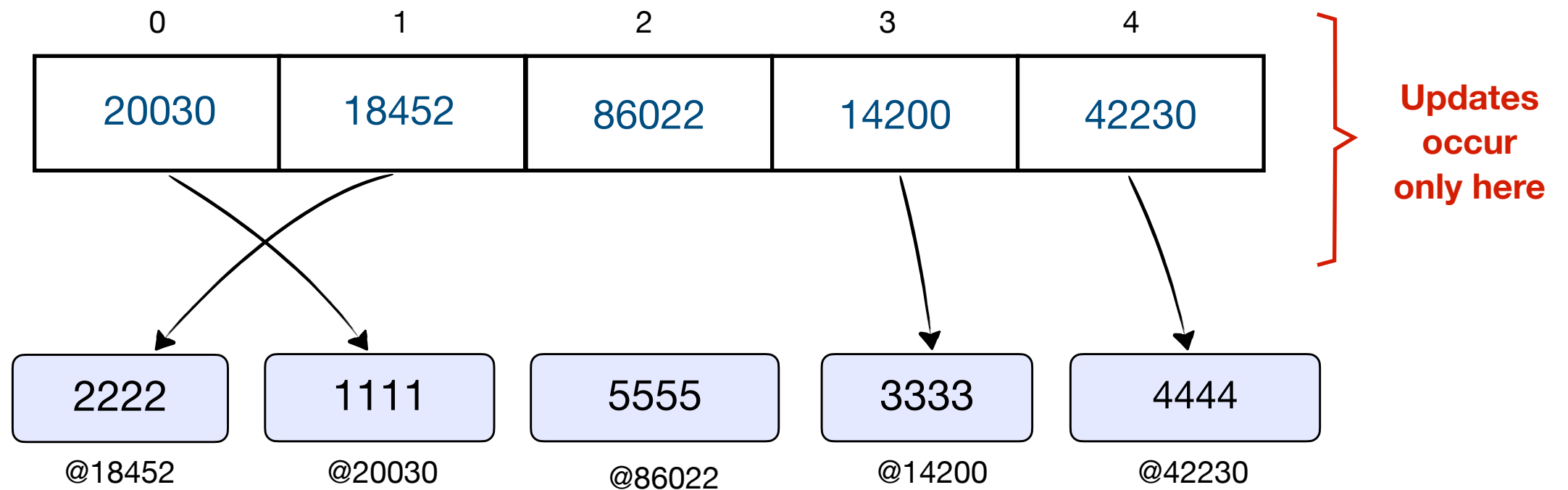
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

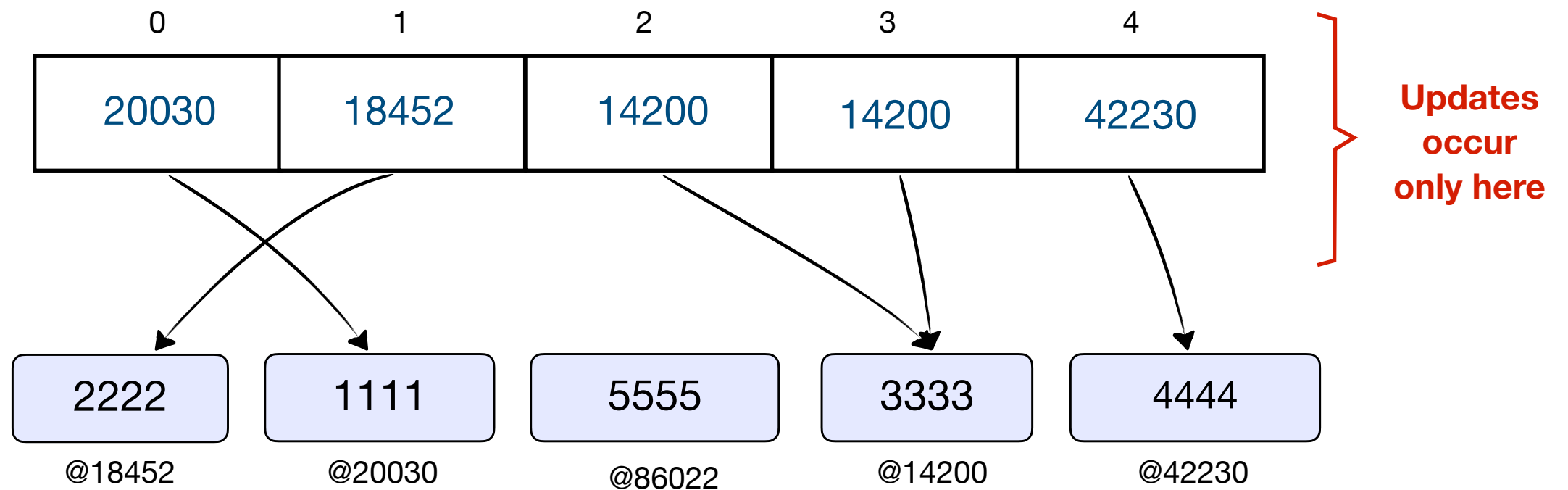
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

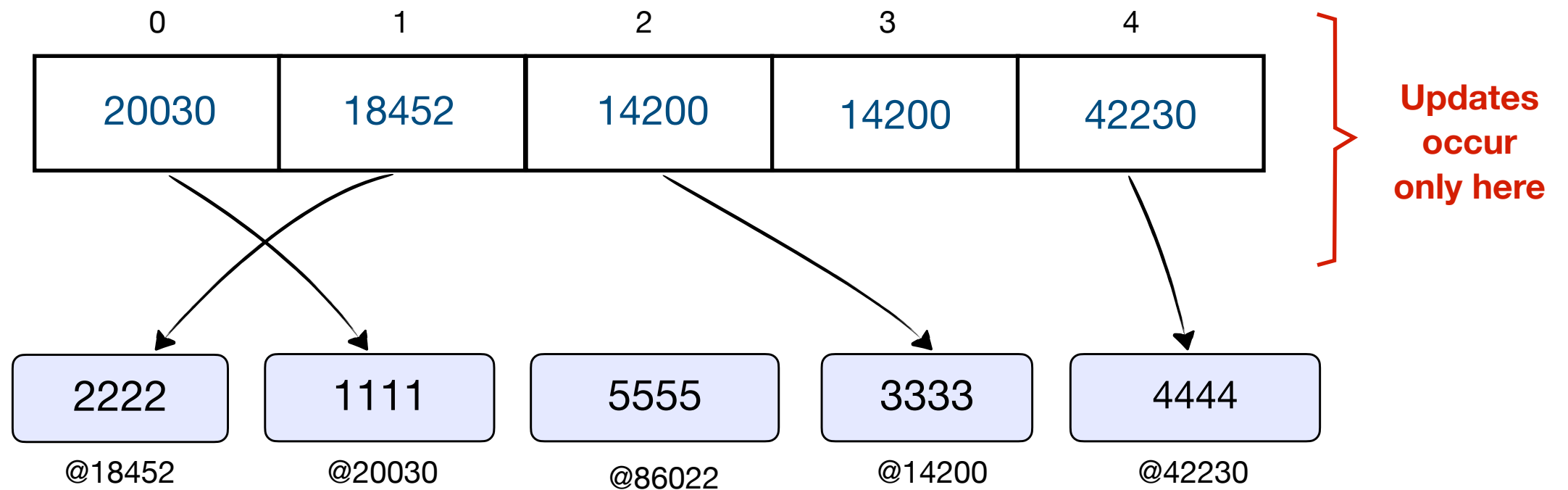
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

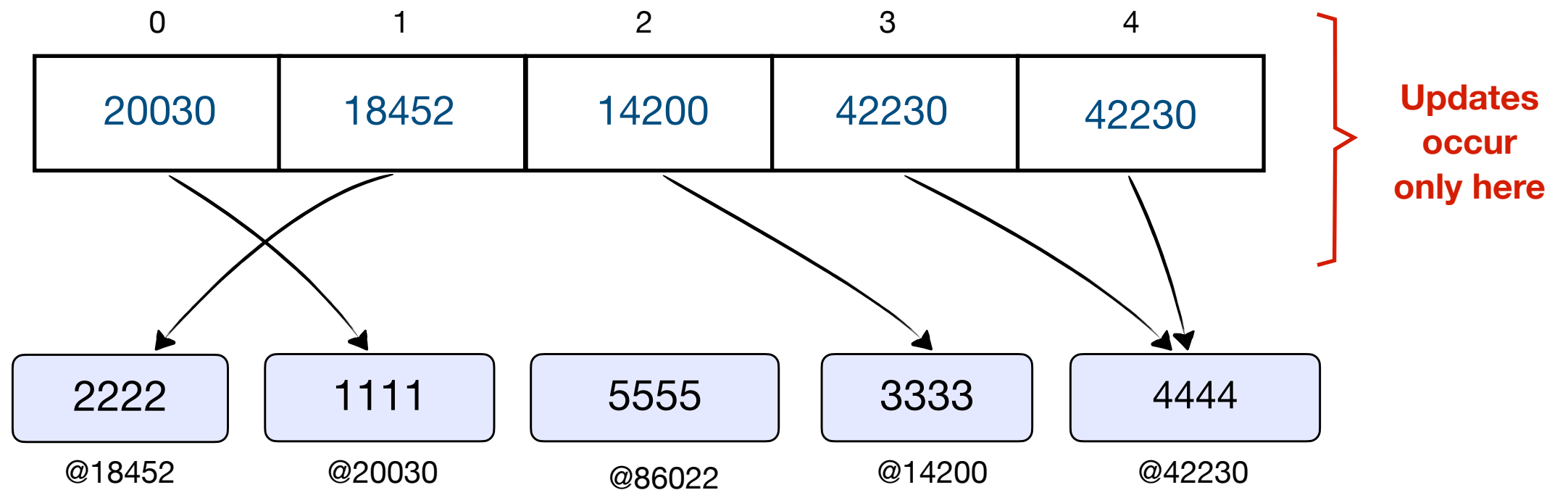
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

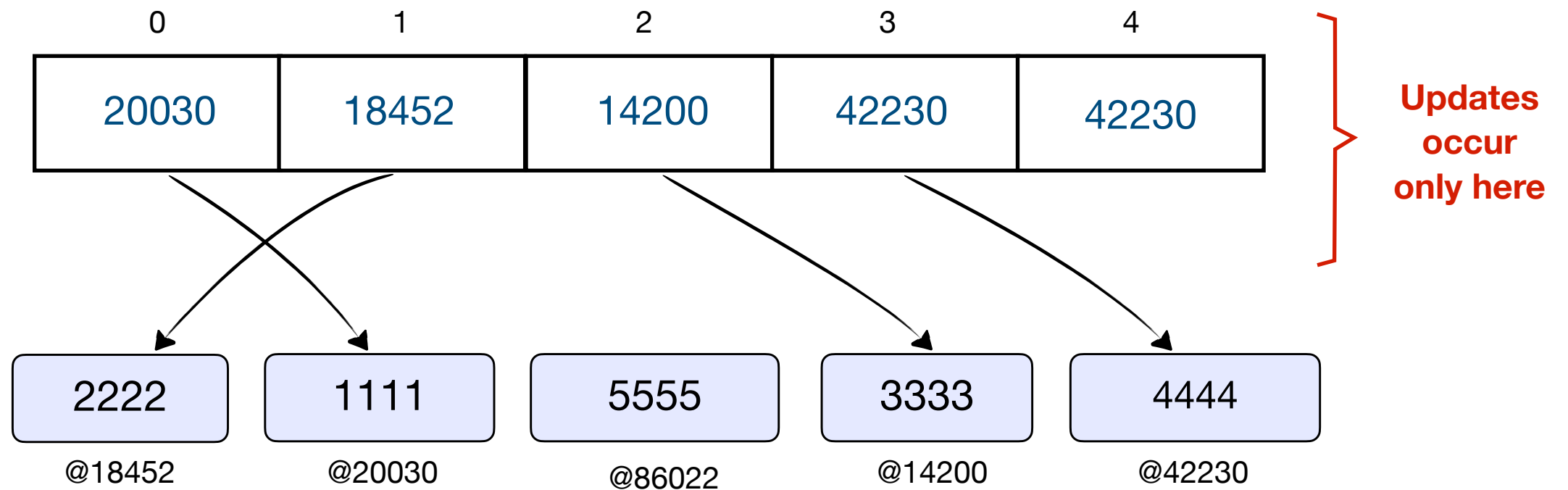
What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

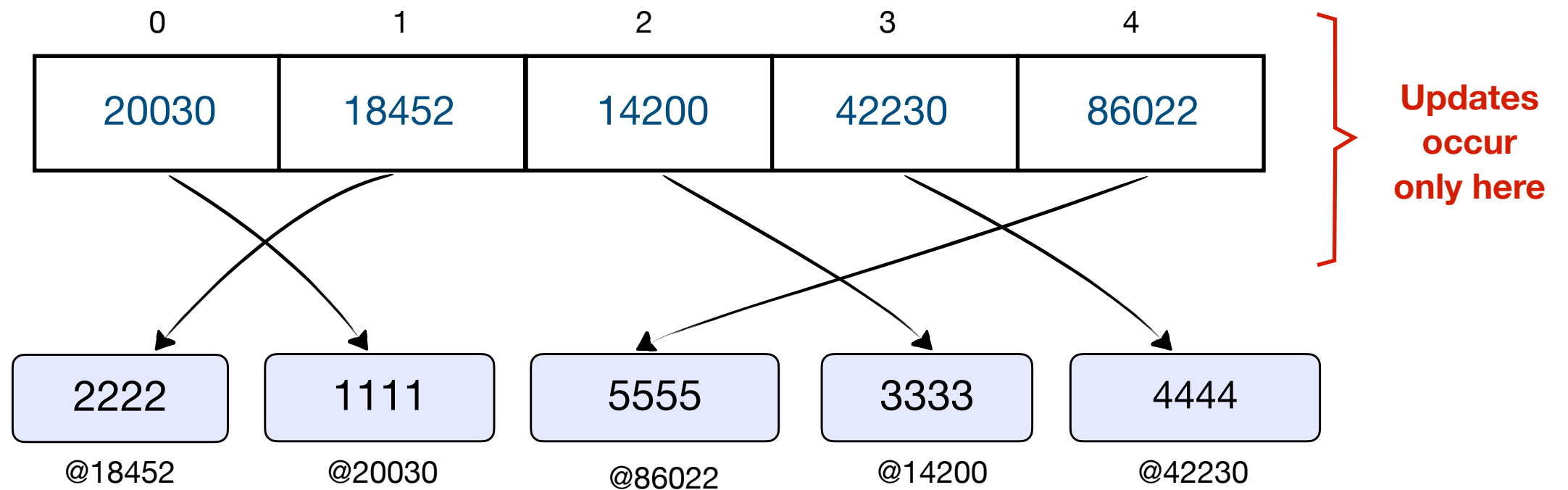

What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```


What happens when we sort a list?



```
L = [2222, 1111, 5555, 3333, 4444]
```

```
L.sort()
```

Side Effects of Aliasing

Side effects of Aliasing

Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

1

@22004

Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

1

@22004

2

@33002

Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

1

@22004

2

@33002

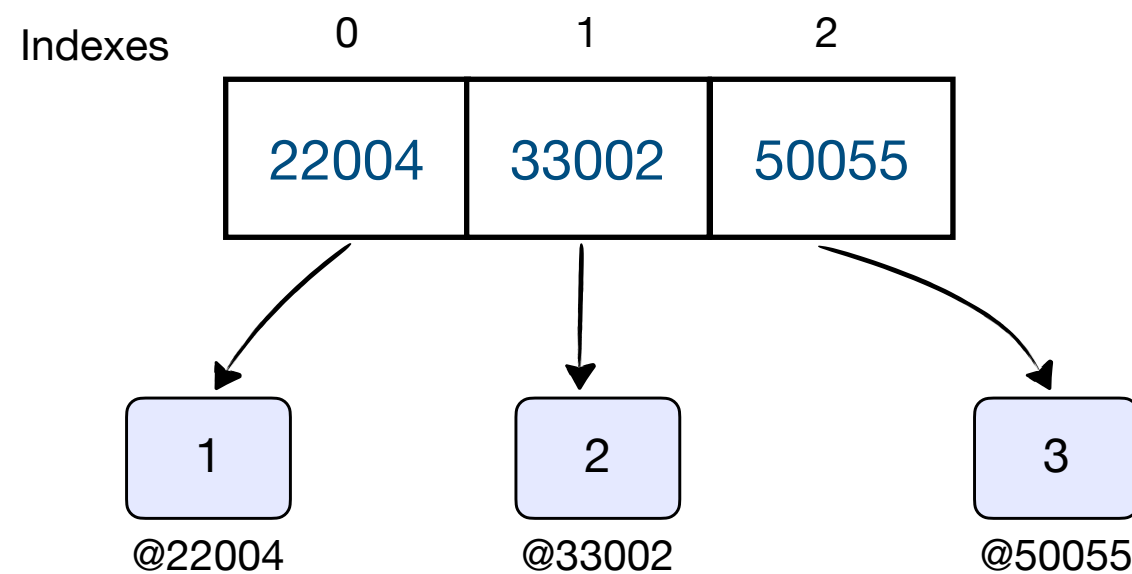
3

@50055

Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

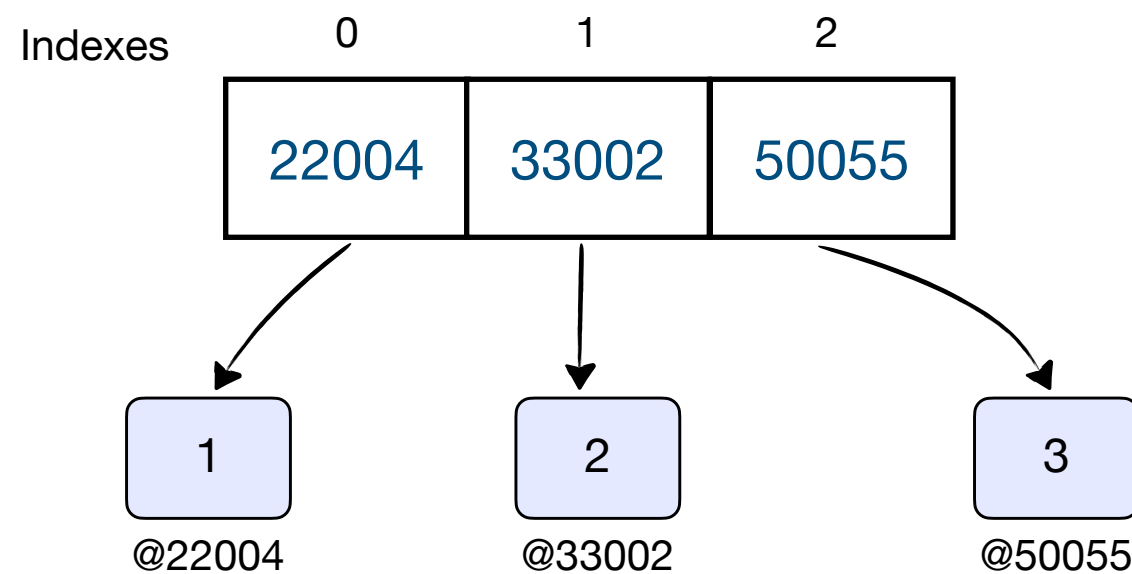


Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

```
L.extend(L)
```

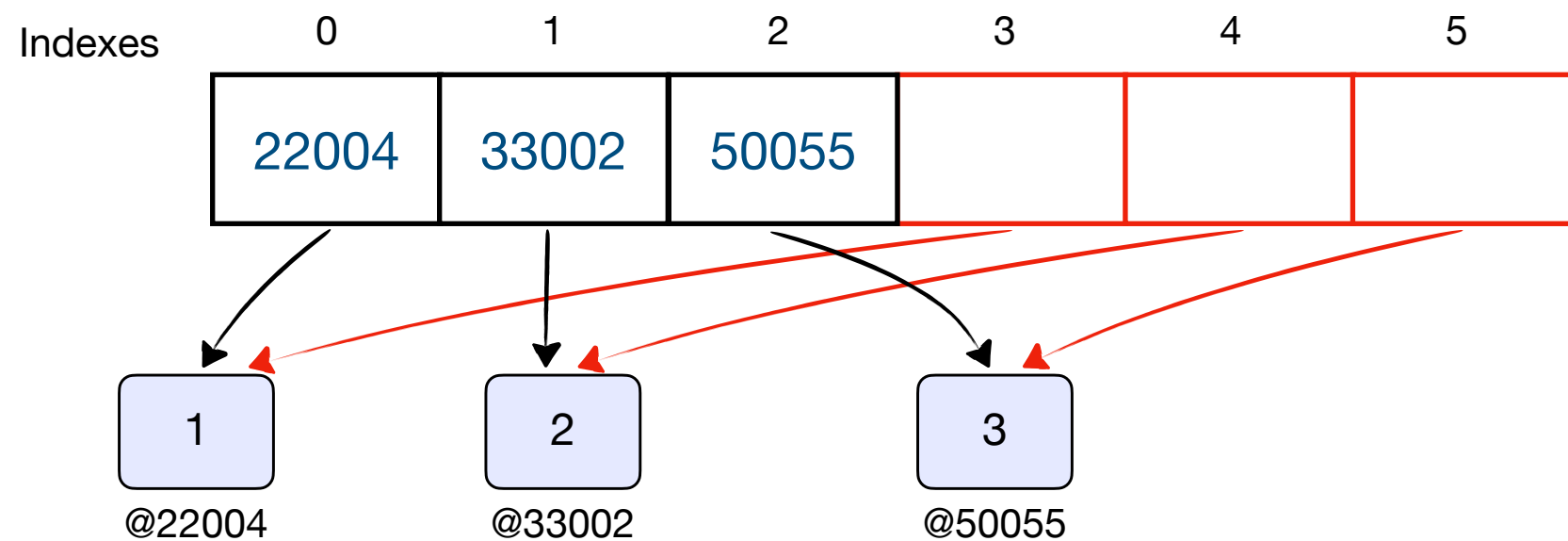


Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

```
L.extend(L)
```



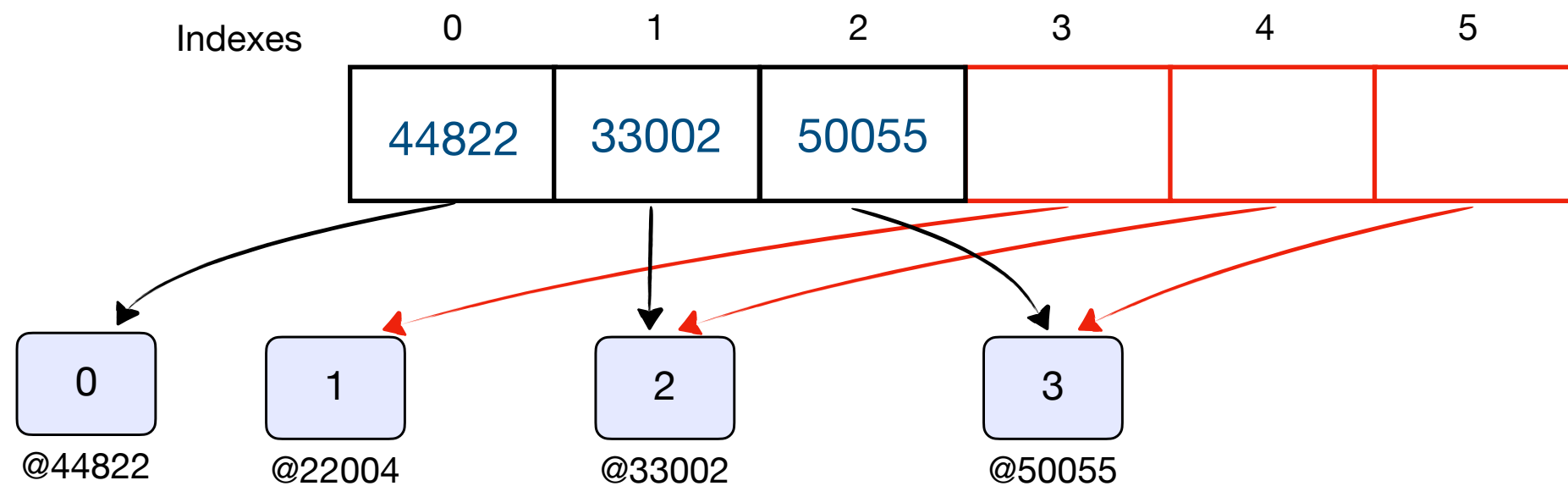
Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

```
L.extend(L)
```

```
L[0] = 0
```



Side effects of Aliasing

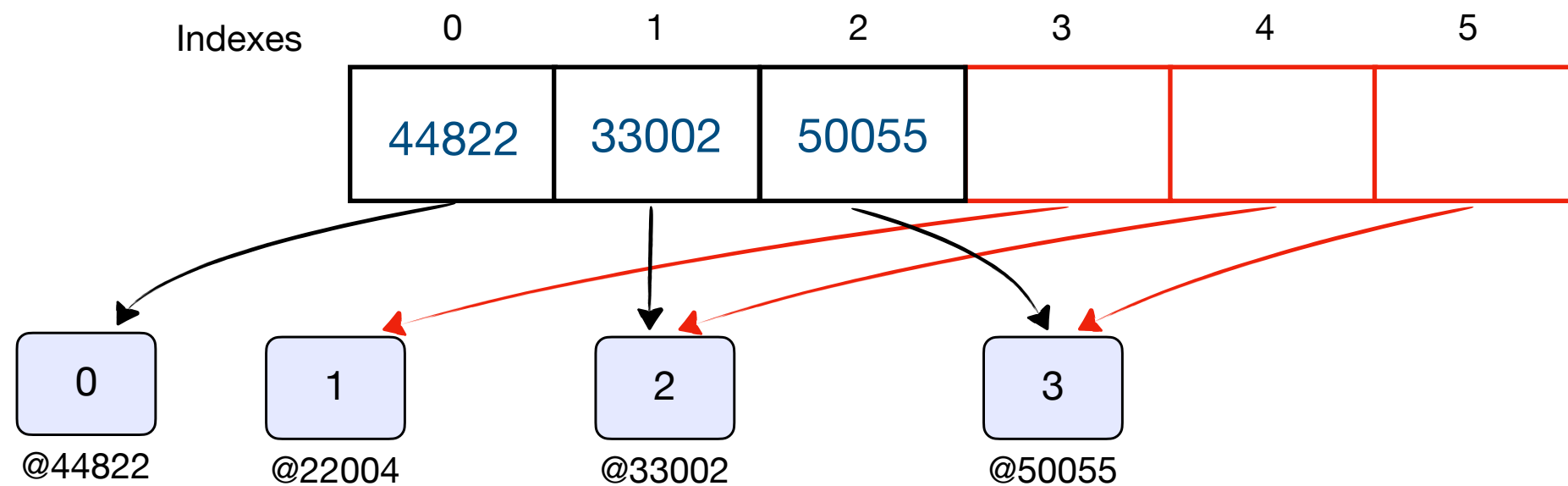
Program

```
L = [1, 2, 3]
```

```
L.extend(L)
```

```
L[0] = 0
```

```
print(L)
```



Side effects of Aliasing

Program

```
L = [1, 2, 3]
```

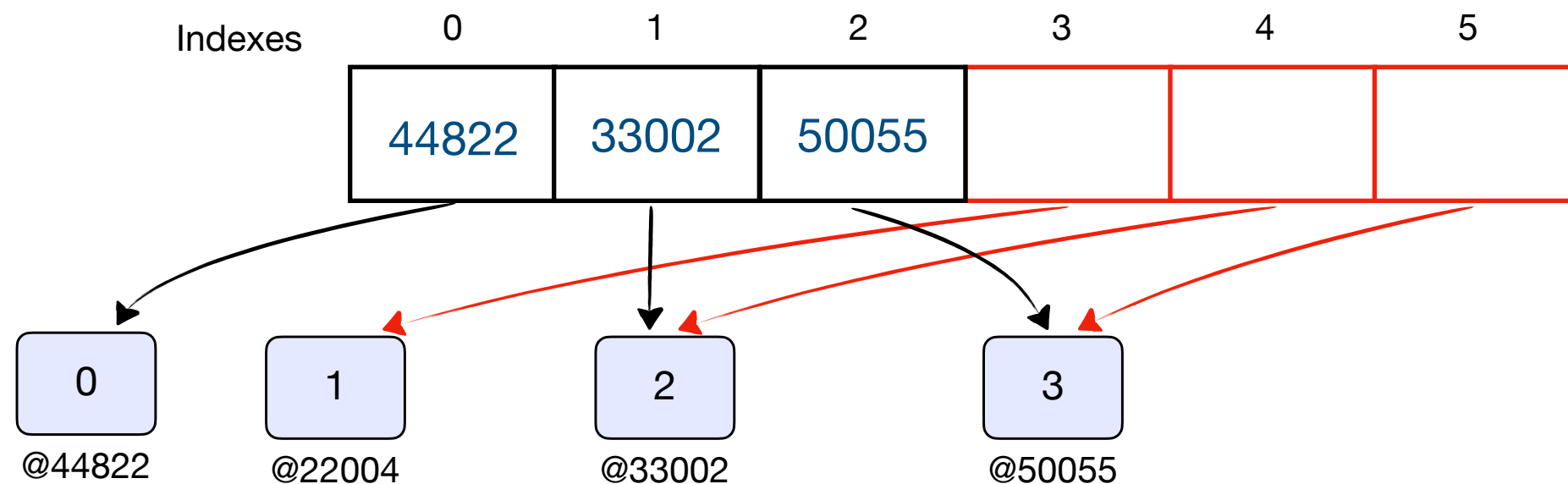
```
L.extend(L)
```

```
L[0] = 0
```

```
print(L)
```

Output

```
[0, 2, 3, 1, 2, 3]
```



Side effects of Aliasing

Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```


Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

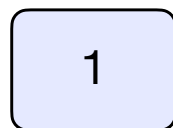
1

@id1

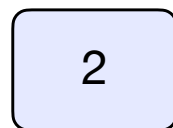
Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```



@id1

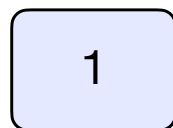


@id2

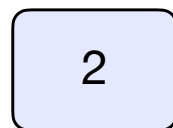
Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```



@id1



@id2

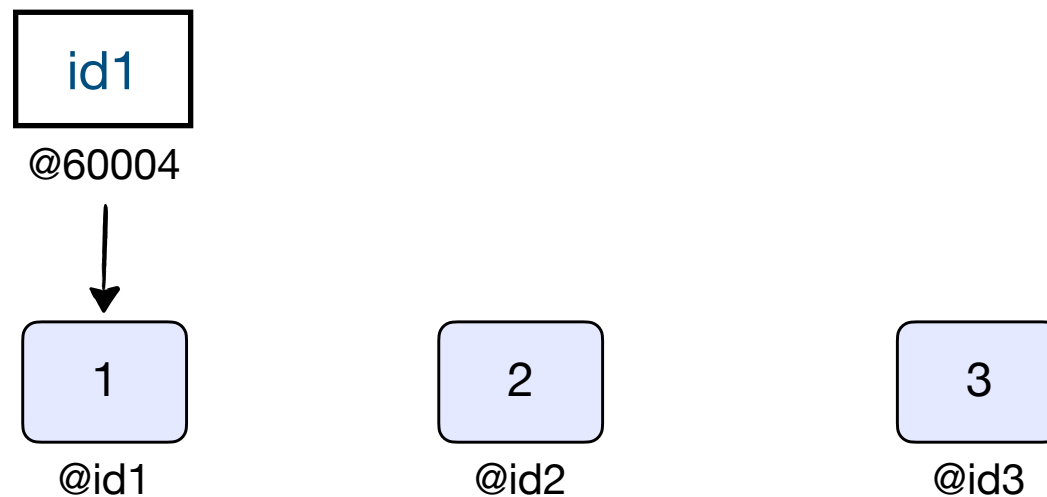


@id3

Side effects of Aliasing

Program

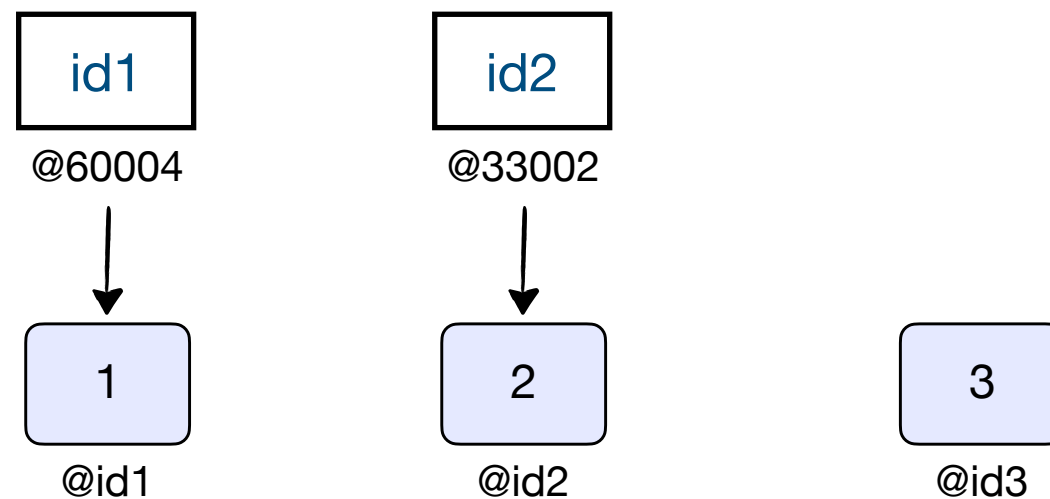
```
L = [[1], [2], [3]]
```



Side effects of Aliasing

Program

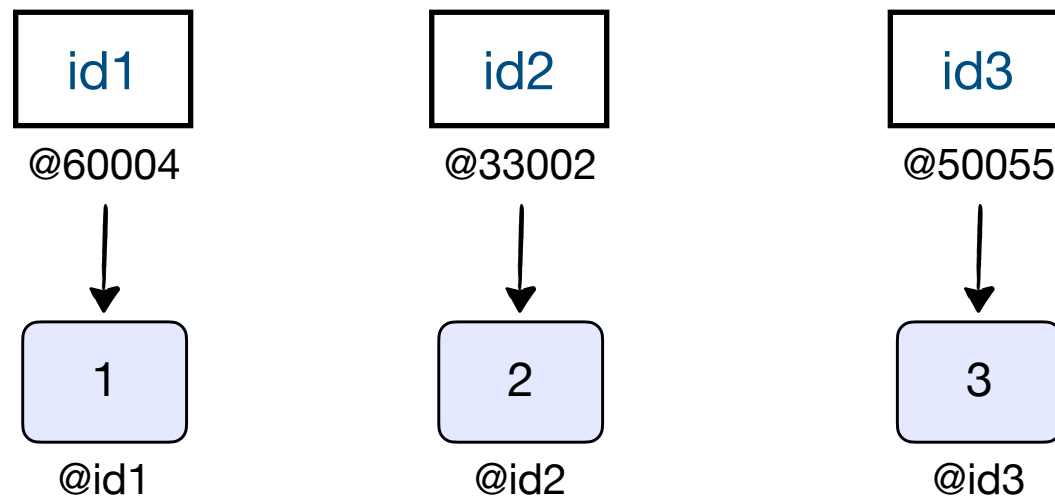
```
L = [[1], [2], [3]]
```



Side effects of Aliasing

Program

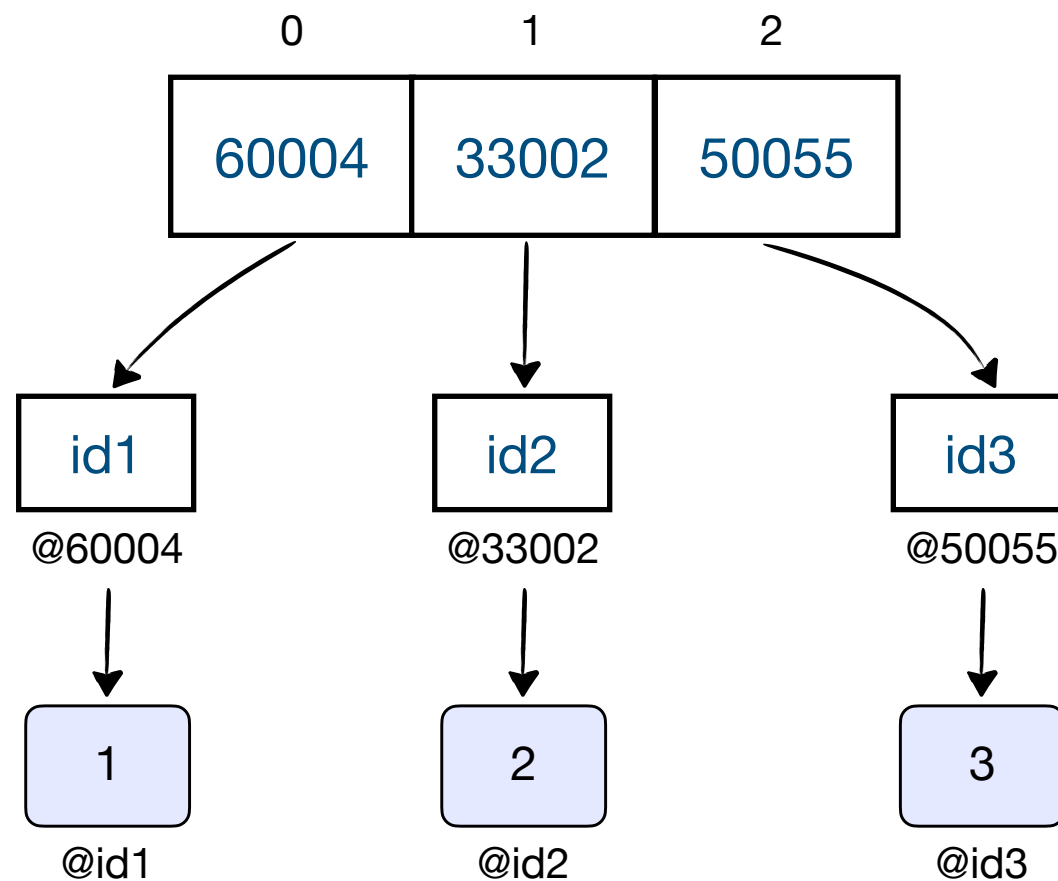
```
L = [[1], [2], [3]]
```



Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

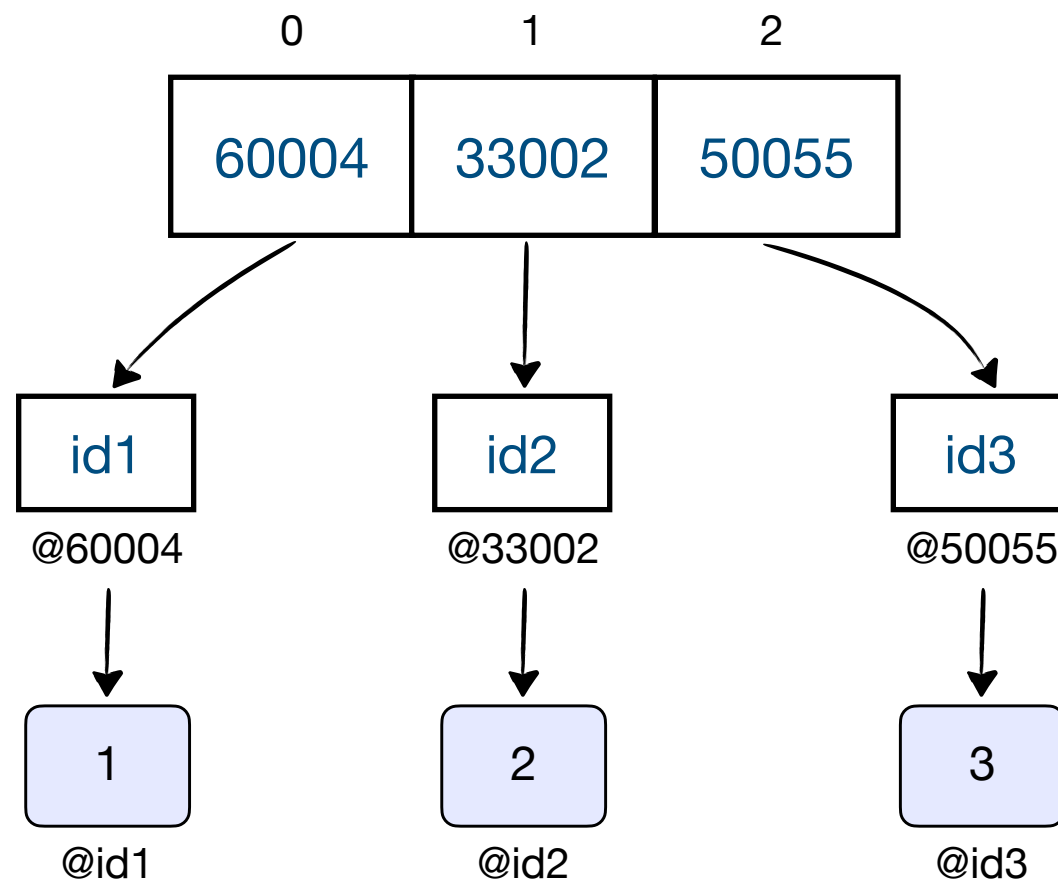


Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

```
L.extend(L)
```

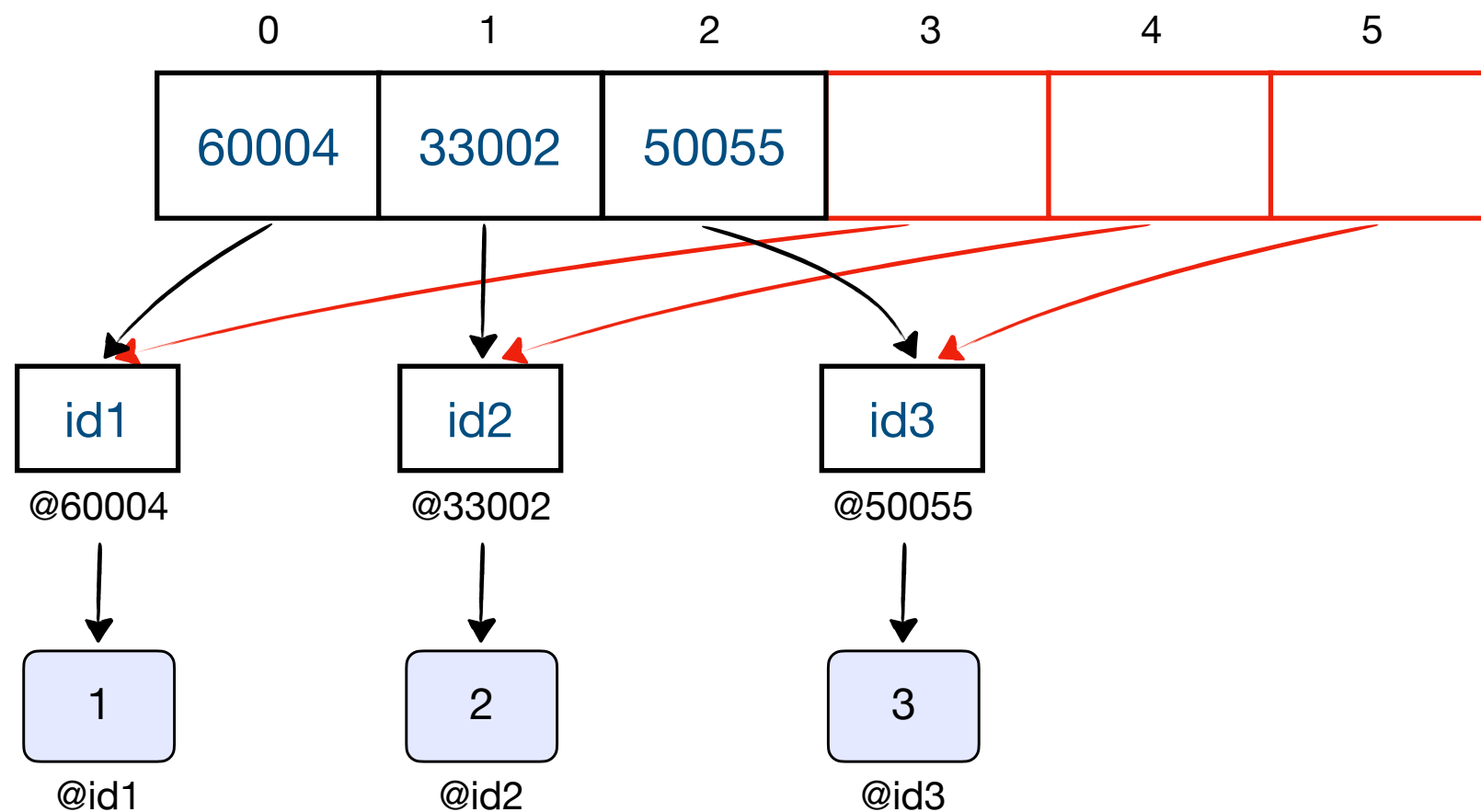


Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

```
L.extend(L)
```



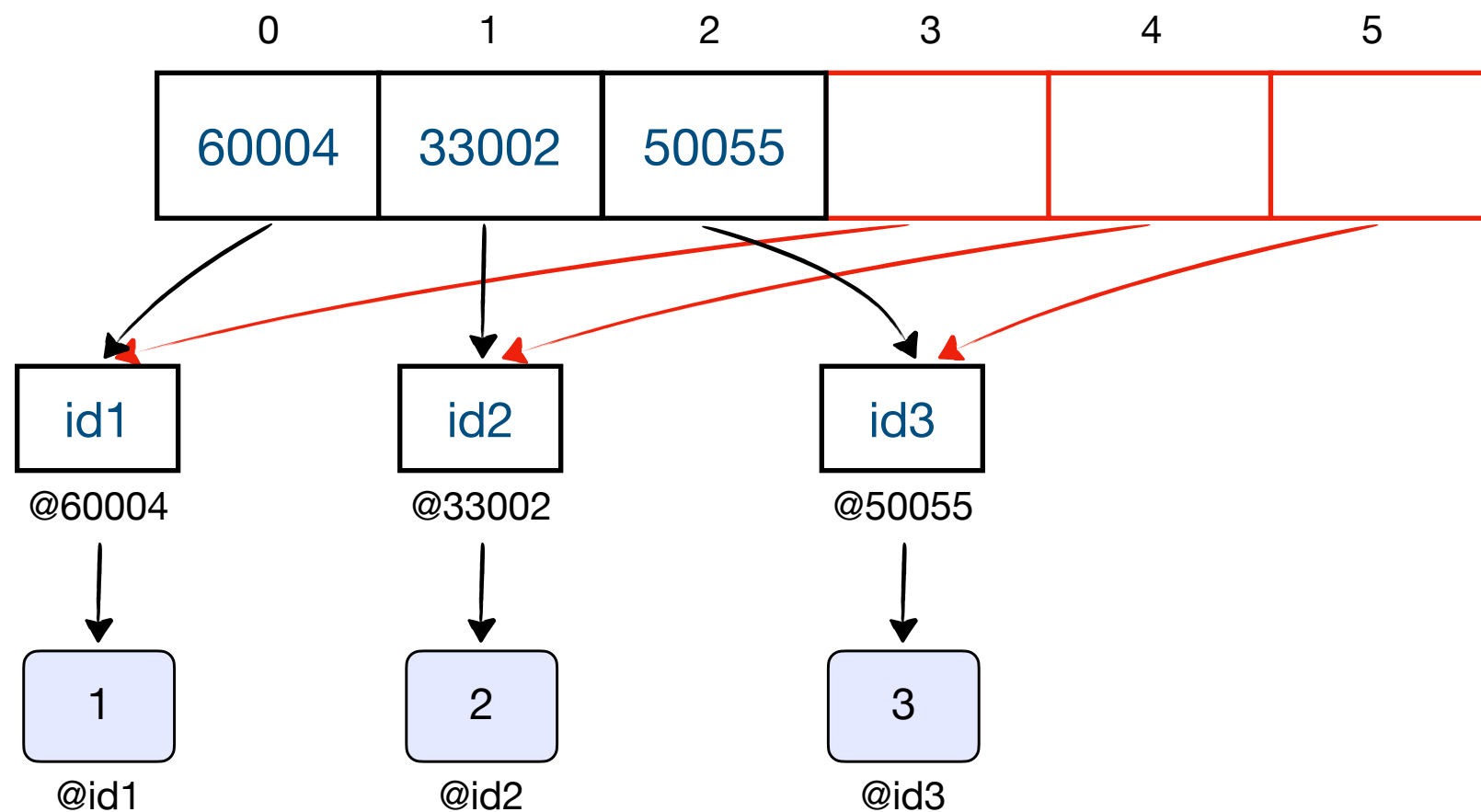
Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

```
L.extend(L)
```

```
L[0][0] = 0
```



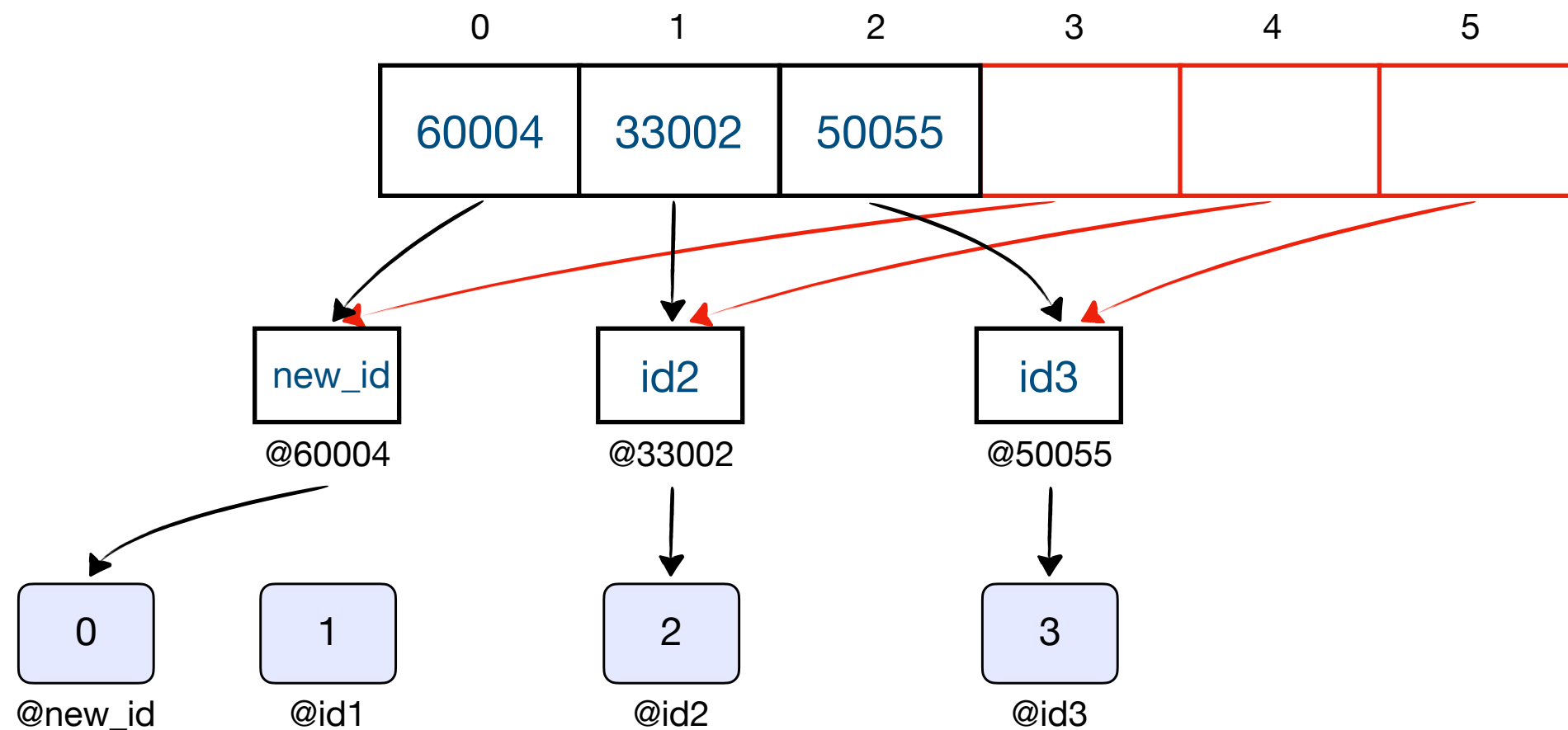
Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

```
L.extend(L)
```

```
L[0][0] = 0
```



Side effects of Aliasing

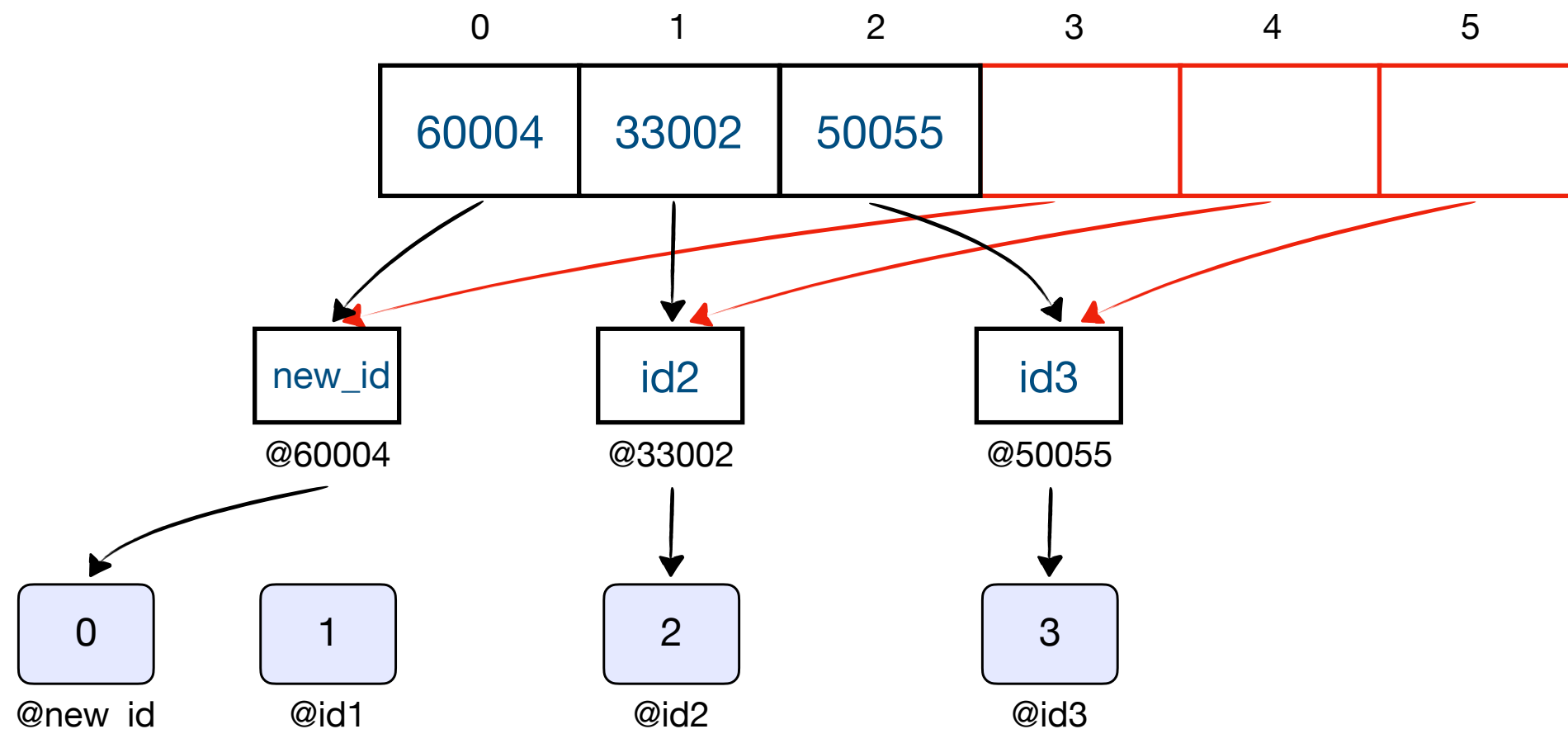
Program

```
L = [[1], [2], [3]]
```

```
L.extend(L)
```

```
L[0][0] = 0
```

```
print(L)
```



Side effects of Aliasing

Program

```
L = [[1], [2], [3]]
```

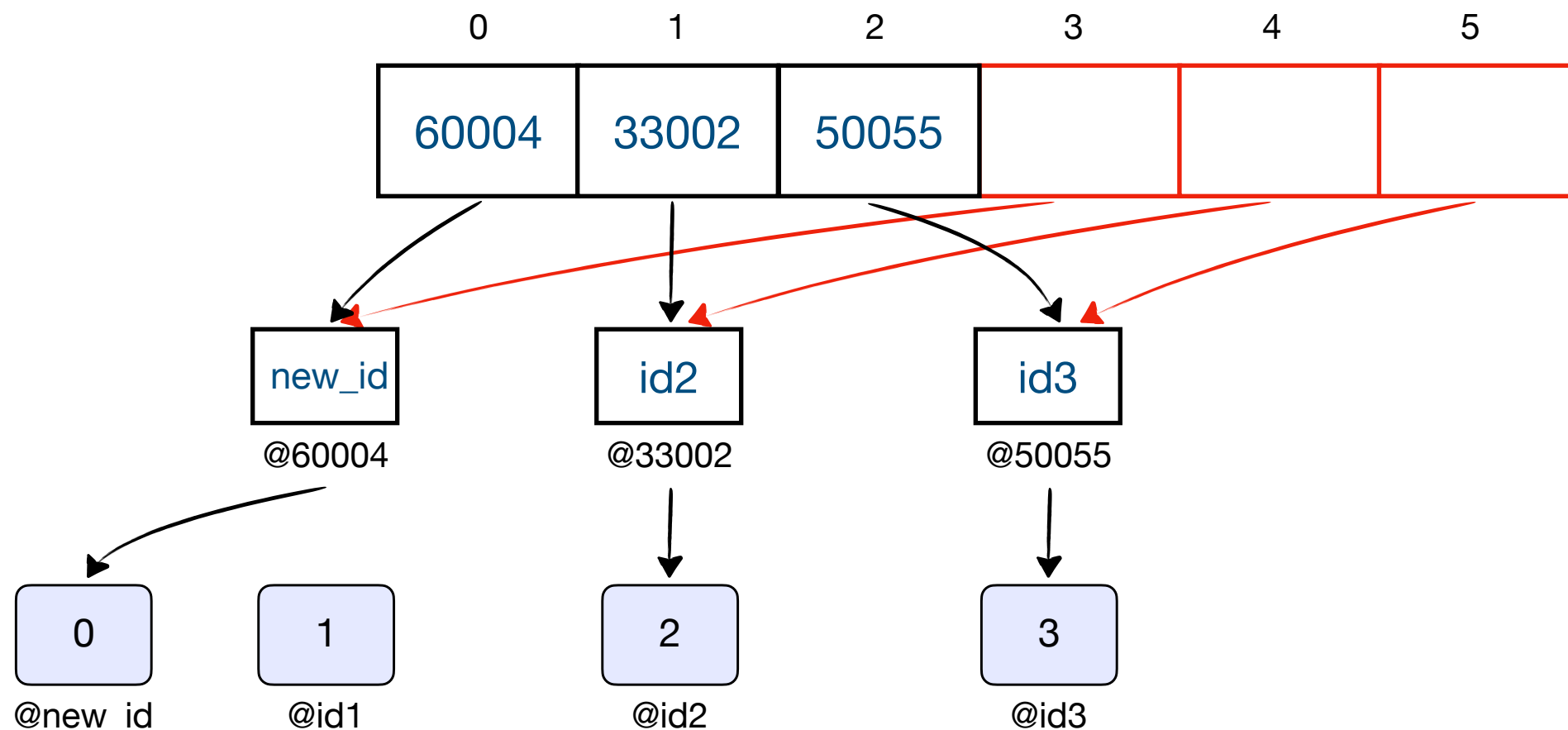
```
L.extend(L)
```

```
L[0][0] = 0
```

```
print(L)
```

Output

```
[[0], [2], [3], [0], [2], [3]]
```



Side effects of Aliasing

Side effects of Aliasing

Program

```
A = [[], []]
```

Side effects of Aliasing

Program

```
A = [[], []]  
B = [[]]*2
```


Side effects of Aliasing

Program

```
A = [], []  
B = []*2  
  
for i in range(2):
```


Side effects of Aliasing

Program

```
A = [], []  
B = []*2  
  
for i in range(2):  
    A[i].append(500+i)
```

Side effects of Aliasing

Program

```
A = [], []  
B = []*2  
  
for i in range(2):  
    A[i].append(500+i)  
    B[i].append(800+i)
```

Side effects of Aliasing

Program

```
A = [], []  
B = [[]]*2  
  
for i in range(2):  
    A[i].append(500+i)  
    B[i].append(800+i)  
  
print("A = ", A)
```

Side effects of Aliasing

Program

```
A = [], []  
B = []*2  
  
for i in range(2):  
    A[i].append(500+i)  
    B[i].append(800+i)  
  
print("A = ", A)  
print("B = ", B)
```

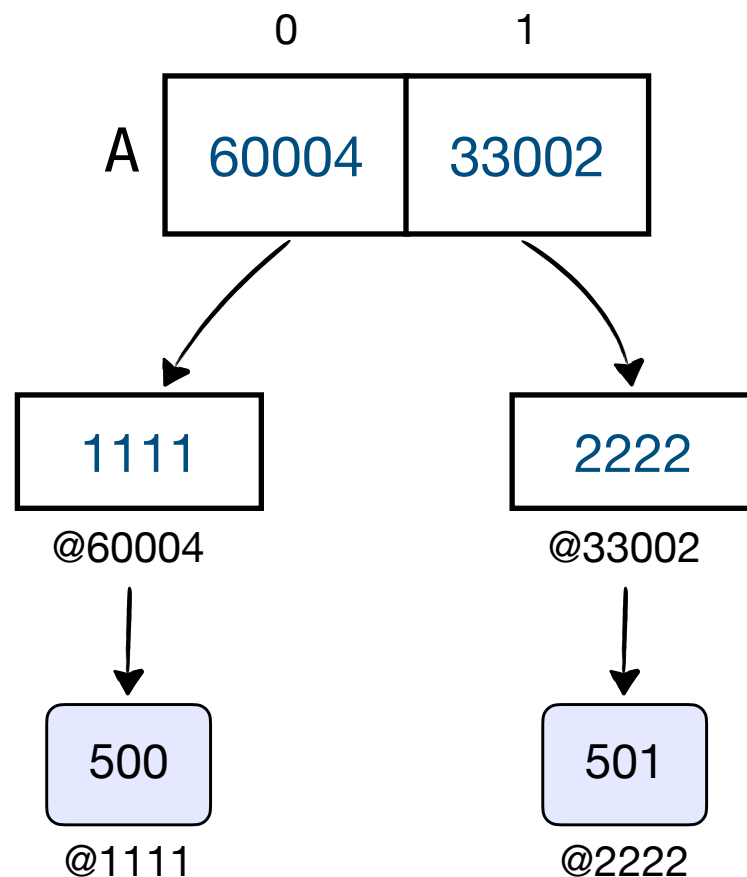
Side effects of Aliasing

Program

```
A = [[], []]
B = [[]]*2

for i in range(2):
    A[i].append(500+i)
    B[i].append(800+i)

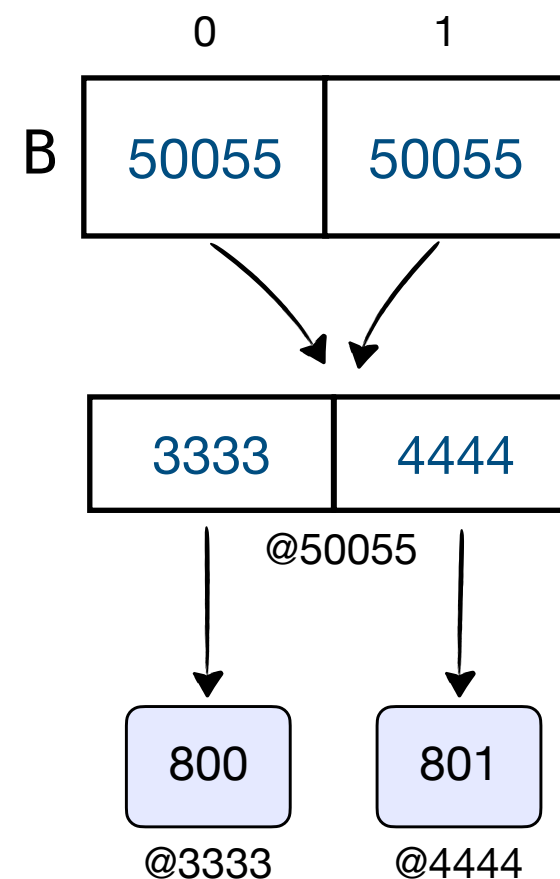
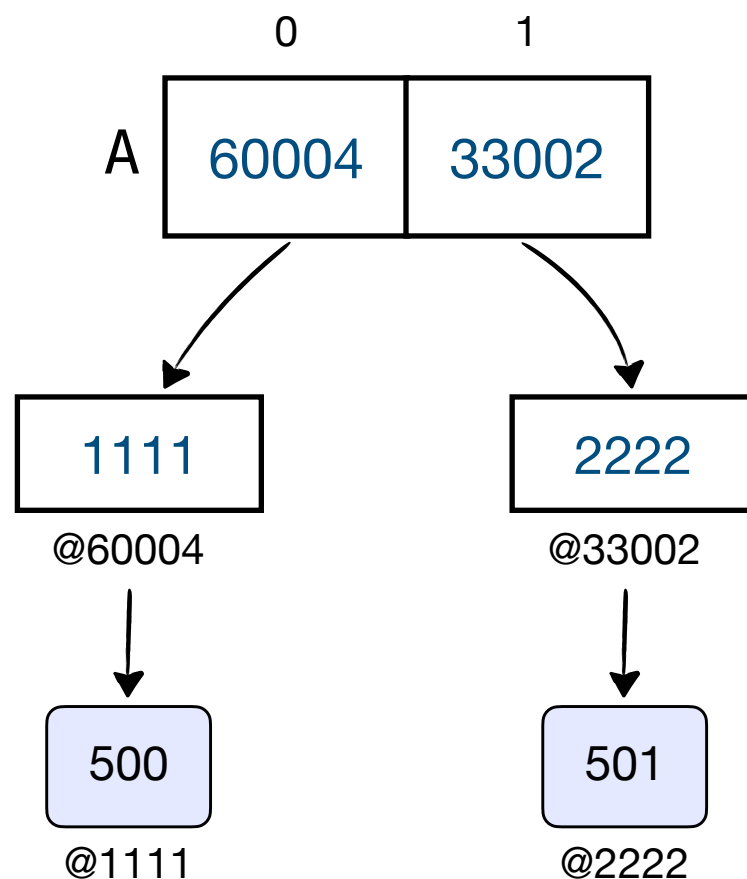
print("A = ", A)
print("B = ", B)
```



Side effects of Aliasing

Program

```
A = [], []  
B = []*2  
  
for i in range(2):  
    A[i].append(500+i)  
    B[i].append(800+i)  
  
print("A = ", A)  
print("B = ", B)
```



Side effects of Aliasing

Program

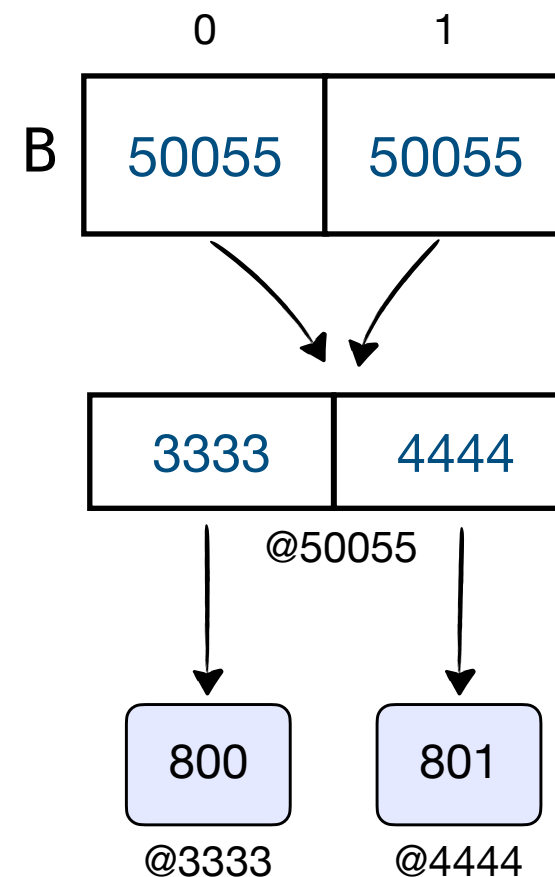
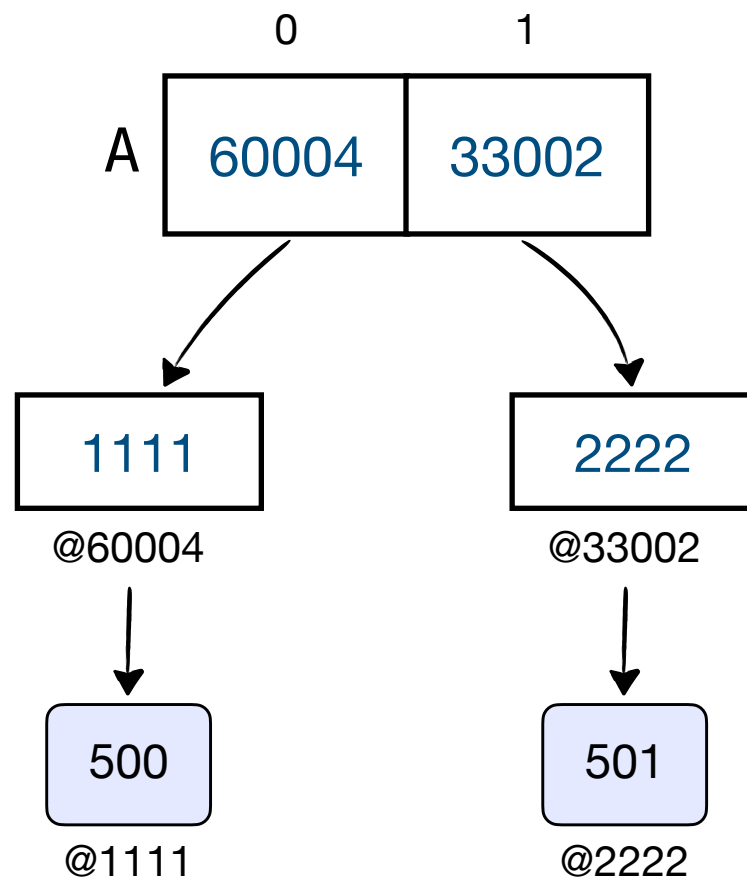
```
A = [[], []]
B = [[]]*2

for i in range(2):
    A[i].append(500+i)
    B[i].append(800+i)

print("A = ", A)
print("B = ", B)
```

Output

```
A = [[500], [501]]
B = [[800, 801], [800, 801]]
```



Side effects of Aliasing

Side effects of Aliasing

Program

```
L = [1, 2, 33, 44]
M = [1, 2, 30, 40]

for e in L:
    if e in M:
        L.remove(e)

print("L = ", L)
print("M = ", M)
```

Side effects of Aliasing

Program

```
L = [1, 2, 33, 44]
M = [1, 2, 30, 40]

for e in L:
    if e in M:
        L.remove(e)

print("L = ", L)
print("M = ", M)
```

Output

```
L = [2, 33, 44]
M = [1, 2, 30, 40]
```

Side effects of Aliasing

Program

```
L = [1, 2, 33, 44]
M = [1, 2, 30, 40]

for e in L:
    if e in M:
        L.remove(e)

print("L = ", L)
print("M = ", M)
```

Output

```
L = [2, 33, 44]
M = [1, 2, 30, 40]
```

Note: when iterating over a list,
be careful with respect to mutating the list

Cloning a List

Cloning a List

Cloning a List

Cloning a list means creating a new, independent copy of an existing list that resides at a different memory location.

Cloning a List

Cloning a list means creating a new, independent copy of an existing list that resides at a different memory location.

Shallow/Surface copy

Creates a new outer list but shares references to inner mutable objects.
Thus, the references to the inner nested objects are retained.

Deep copy

Recursively duplicates, creating a fully clone of mutable objects .

Cloning a List

Cloning a list means creating a new, independent copy of an existing list that resides at a different memory location.

Shallow/Surface copy

Creates a new outer list but shares references to inner mutable objects.
Thus, the references to the inner nested objects are retained.

Deep copy

Recursively duplicates, creating a fully clone of mutable objects .

Note: Changes to mutable inner lists affect shallow copies but not deep copies.

Cloning a List

Cloning a List

Program

```
A = [500, 501, 502]
```

```
B = A[:]
```

```
print(id(A))
```

```
print(id(B))
```

Cloning a List

Program

```
A = [500, 501, 502]
```

```
B = A[:]
```

```
print(id(A))
```

```
print(id(B))
```

Output

```
112233445566789
```

Cloning a List

Program

```
A = [500, 501, 502]
```

```
B = A[:]
```

```
print(id(A))
```

```
print(id(B))
```

Output

```
112233445566789
```

```
112233445500009
```

Cloning a List

Program

```
A = [500, 501, 502]

B = A[:]

print(id(A))
print(id(B))
```

Output

```
112233445566789
112233445500009
```

`B = A[ : ]` creates a **shallow** copy of the list

Cloning a List

Cloning a List

Program

```
A = [500, 501, 502]
```

```
B = A.copy()
```

```
B[0] = 999
```

```
print("A = ", A)
```

```
print("B = ", B)
```


Cloning a List

Program

```
A = [500, 501, 502]
```

```
B = A.copy()
```

```
B[0] = 999
```

```
print("A = ", A)
```

```
print("B = ", B)
```

Output

Cloning a List

Program

```
A = [500, 501, 502]

B = A.copy()
B[0] = 999

print("A = ", A)
print("B = ", B)
```

Output

```
A = [500, 501, 502]
```

`B = A.copy()` also creates a **shallow** copy of the list

Cloning a List

Program

```
A = [500, 501, 502]

B = A.copy()
B[0] = 999

print("A = ", A)
print("B = ", B)
```

Output

```
A = [500, 501, 502]
B = [999, 501, 502]
```

`B = A.copy()` also creates a **shallow** copy of the list

Cloning a List

Cloning a List

Program

```
A = [[500], [501], [502]]
```

```
B = A.copy()
```

```
B[0][0] = 999
```

```
print("A = ", A)
```

```
print("B = ", B)
```

Cloning a List

Program

```
A = [[500], [501], [502]]  
  
B = A.copy()  
B[0][0] = 999  
  
print("A = ", A)  
print("B = ", B)
```

Output

```
A = [[999], [501], [502]]
```

Cloning a List

Program

```
A = [[500], [501], [502]]  
  
B = A.copy()  
B[0][0] = 999  
  
print("A = ", A)  
print("B = ", B)
```

Output

```
A = [[999], [501], [502]]  
B = [[999], [501], [502]]
```

Cloning a List

Program

```
A = [[500], [501], [502]]  
  
B = A.copy()  
B[0][0] = 999  
  
print("A = ", A)  
print("B = ", B)
```

Output

```
A = [[999], [501], [502]]  
B = [[999], [501], [502]]
```

To create completely independent copies,
use **copy.deepcopy(L)** under copy library.

Cloning a List

Cloning a List

Program

```
import copy

A = [[500], [501], [502]]

B = copy.deepcopy(A)

B[0][0] = 999

print("A = ", A)
print("B = ", B)
```

Cloning a List

Program

```
import copy

A = [[500], [501], [502]]

B = copy.deepcopy(A)

B[0][0] = 999

print("A = ", A)
print("B = ", B)
```

Output

```
A = [[500], [501], [502]]
```

Cloning a List

Program

```
import copy  
  
A = [[500], [501], [502]]  
  
B = copy.deepcopy(A)  
  
B[0][0] = 999  
  
print("A = ", A)  
print("B = ", B)
```

Output

```
A = [[500], [501], [502]]  
B = [[999], [501], [502]]
```

Cloning a List

Program

```
import copy  
  
A = [[500], [501], [502]]  
  
B = copy.deepcopy(A)  
  
B[0][0] = 999  
  
print("A = ", A)  
print("B = ", B)
```

Output

```
A = [[500], [501], [502]]  
B = [[999], [501], [502]]
```

Note: A is unaffected by changes in B

Invariants

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]
```


Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]  
curr_max = A[0]
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]

print(curr_max)
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

Invariant 1:

When iteration i starts,

`curr_max` stores max of
`A[0], ..., A[i-1]`

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]

print(curr_max)
```

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

Invariant 1:

When iteration i starts,

`curr_max` stores max of
`A[0], ..., A[i-1]`

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]

print(curr_max)
```

Invariant 1 holds on
entering the loop

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

Invariant 1:

When iteration i starts,

`curr_max` stores max of
 $A[0], \dots, A[i-1]$

Invariant 2:

When iteration i ends,

`curr_max` stores max of
 $A[0], \dots, A[i]$

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]

print(curr_max)
```

Invariant 1 holds on
entering the loop

Invariants

An **invariant** is a *condition* that is *guaranteed to be true at specific points* during a code's execution. They are helpful in reasoning correctness.

Example:

Computing max
element of a list

Invariant 1:

When iteration i starts,

`curr_max` stores max of
 $A[0], \dots, A[i-1]$

Invariant 2:

When iteration i ends,

`curr_max` stores max of
 $A[0], \dots, A[i]$

```
A = [3, 5, -1, 4, 0]
curr_max = A[0]

for i in range(1, len(A)):

    if (A[i] > curr_max):
        curr_max = A[i]

print(curr_max)
```

Invariant 1 holds on
entering the loop

Claim: Invariant 2
ensures that the final
output is correct

Searching in Sorted Lists

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
```

```
x = 12
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
```

```
x = 12
```

```
left = 0
```

```
right = len(L) - 1
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
```

```
x = 12
```

```
left = 0
```

```
right = len(L) - 1
```

```
found = False
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
```

```
x = 12
```

```
left = 0
```

```
right = len(L) - 1
```

```
found = False
```

```
while (left <= right):
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2
```


Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
```


Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): _____
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L, then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L, then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Invariant 1 holds on entering the loop

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right) // 2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L, then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Invariant 1 holds on entering the loop

Check:

Invariant 1 holds

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L, then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Invariant 1 holds on entering the loop

Check:

Invariant 1 holds

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right)//2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L , then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Claim 1: Invariant 1 ensures that the output is correct (if the code terminates).

Check:
Invariant 1 holds

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right) // 2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L , then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Claim 1: Invariant 1 ensures that the output is correct (if the code terminates).

Invariant 2:

$(\text{right} - \text{left})$ decreases monotonically with each iteration.

Check:

Invariant 1 holds

Searching in Sorted Lists

```
L = [0, 2, 4, 8, 12, 16]
x = 12
left = 0
right = len(L) - 1
found = False

while (left <= right):

    mid = (left + right) // 2

    if (x == L[mid]):
        found = True
        break

    if (x > L[mid]): left = mid + 1

    if (x < L[mid]): right = mid - 1
```

Invariant 1 (anywhere inside loop):

If x is in L , then

- $\text{left} \leq \text{right}$
- $L[\text{left}] \leq x \leq L[\text{right}]$

Claim 1: Invariant 1 ensures that the output is correct (if the code terminates).

Invariant 2:

$(\text{right} - \text{left})$ decreases monotonically with each iteration.

Claim 2: Invariant 2 ensures that the code terminates.

Check:

Invariant 1 holds

Generating Binary Strings of length m

Generating Binary Strings of length m

Sub-problem: Create a list L of size $m+1$, where, $L[i]$ itself is a list storing binary strings of length i

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, $L[i]$ itself is a list storing binary strings of length i

m = 10

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10  
L = [ [""], ["0", "1"] ]
```


Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [""], ["0", "1"] ]

for i in range(2, m+1):
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [""], ["0", "1"] ]

for i in range(2, m+1):

    result = []
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [""], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [""], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [""], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)
```

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)
```


Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)

print(L[m])
```


Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Invariant 2:

In iteration for index i, on termination of the inner loop,

result stores all i-length binary strings.

Generating Binary Strings of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing binary strings of length i

```
m = 10
L = [ [], ["0", "1"] ]

for i in range(2, m+1):

    result = []

    for s in L[i-1]:
        result.append("0" + s)
        result.append("1" + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Invariant 2:

In iteration for index i, on termination of the inner loop,

result stores all i-length binary strings.

Claim: Invariants 1, 2 ensure that the output is correct.

Ordered Partitions of length m

Ordered Partitions of length m

Sub-problem: Create a list L of size $m+1$, where, $L[i]$ itself is a list storing ordered partitions of i

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, $L[i]$ itself is a list storing ordered partitions of i

m = 10

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
```

```
L = [ [""], ["1"], ["1 1", "2"] ]
```


Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, $L[i]$ itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):
```


Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []
```

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
```

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
```

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j)+" "+s)
```

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j)+" "+s)

    L.append(result)

print(L[m])
```

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j) + " " + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, $L[i]$ itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j) + " " + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- $L[0], \dots, L[i-1]$ are correctly computed.

Invariant 1 holds on entering the loop

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j) + " " + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Invariant 2:

In iteration for index i, on termination of the inner loop,

S stores ALL partitions of i starting with 1,2,...,j

Ordered Partitions of length m

Sub-problem: Create a list L of size m+1, where, L[i] itself is a list storing ordered partitions of i

```
m = 10
L = [ [], ["1"], ["1 1", "2"] ]

for i in range(3, m+1):

    result = []

    for j in range(1, i+1):
        for s in L[i - j]:
            result.append(str(j) + " " + s)

    L.append(result)

print(L[m])
```

Invariant 1:

When iteration for index i starts:

- L has length i, and
- L[0], ..., L[i-1] are correctly computed.

Invariant 2:

In iteration for index i, on termination of the inner loop,

S stores ALL partitions of i starting with 1,2,...,j

Claim: Invariants 1, 2 ensure that the output is correct.

Invariants

(Recursive Function calls)

Generating Binary Strings of length m

Generating Binary Strings of length m

Recursive approach:

```
def binary_strings(i):  
    if (i == 0): return [""]  
  
    result = []  
  
    previous = binary_strings(i - 1)  
  
    for s in previous:  
        result.append("0" + s)  
        result.append("1" + s)  
  
    return result  
  
m = 10  
print(binary_strings(m))
```

Generating Binary Strings of length m

Recursive approach:

```
def binary_strings(i):  
    if (i == 0): return [""]  
  
    result = []  
  
    previous = binary_strings(i - 1)  
  
    for s in previous:  
        result.append("0" + s)  
        result.append("1" + s)  
  
    return result  
  
m = 10  
print(binary_strings(m))
```

Invariant 1:

Function `binary_strings(i)` returns a list storing ALL binary strings of length `i`..

Generating Binary Strings of length m

Recursive approach:

```
def binary_strings(i):  
    if (i == 0): return [""]  
  
    result = []  
  
    previous = binary_strings(i - 1)  
  
    for s in previous:  
        result.append("0" + s)  
        result.append("1" + s)  
  
    return result  
  
m = 10  
print(binary_strings(m))
```

Invariant 1:

Function `binary_strings(i)` returns a list storing ALL binary strings of length `i`..

Invariant 1 holds for `i = 0`

Invariant 2:

On termination of the loop in function call `binary_strings(i)`,

`result` stores all `i`-length binary strings.

Generating Binary Strings of length m

Recursive approach:

```
def binary_strings(i):  
    if (i == 0): return [""]  
  
    result = []  
  
    previous = binary_strings(i - 1)  
  
    for s in previous:  
        result.append("0" + s)  
        result.append("1" + s)  
  
    return result  
  
m = 10  
print(binary_strings(m))
```

Invariant 1:

Function `binary_strings(i)` returns a list storing ALL binary strings of length `i`..

Invariant 1 holds for `i = 0`

Invariant 2:

On termination of the loop in function call `binary_strings(i)`,

`result` stores all `i`-length binary strings.

Claim: Invariant 1 ensure that the output is correct.

Ordered Partitions of length m

Ordered Partitions of length m

Recursive approach:

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
m = 5  
compute([], m)
```

Ordered Partitions of length m

Recursive approach:

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
m = 5  
compute([], m)
```

Invariant:

`Compute(prefix, m)` generates all lists `L` of +ve integers summing to `m` and prints each as `prefix + L`.

Ordered Partitions of length m

Recursive approach:

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)
```

```
m = 5  
compute([], m)
```

Invariant:

`Compute(prefix, m)` generates all lists `L` of +ve integers summing to `m` and prints each as `prefix + L`.

Base case ($m = 0$) holds.

Ordered Partitions of length m

Recursive approach:

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
m = 5  
compute([], m)
```

Invariant:

`Compute(prefix, m)` generates all lists L of +ve integers summing to m and prints each as `prefix + L`.

Base case ($m = 0$) holds.

Inductive step holds as by induction, for each possible first element $i \in [1, m]$,

`compute(prefix+[i], m-i)`

correctly prints all lists of form `prefix + L` where $L[0] = i$ and remaining elements sum to $m-i$.

Ordered Partitions of length m

Recursive approach:

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
m = 5  
compute([], m)
```

Invariant:

`Compute(prefix, m)` generates all lists L of +ve integers summing to m and prints each as `prefix + L`.

Base case ($m = 0$) holds.

Inductive step holds as by induction, for each possible first element $i \in [1, m]$,

`compute(prefix+[i], m-i)`

correctly prints all lists of form `prefix + L` where $L[0] = i$ and remaining elements sum to $m-i$.

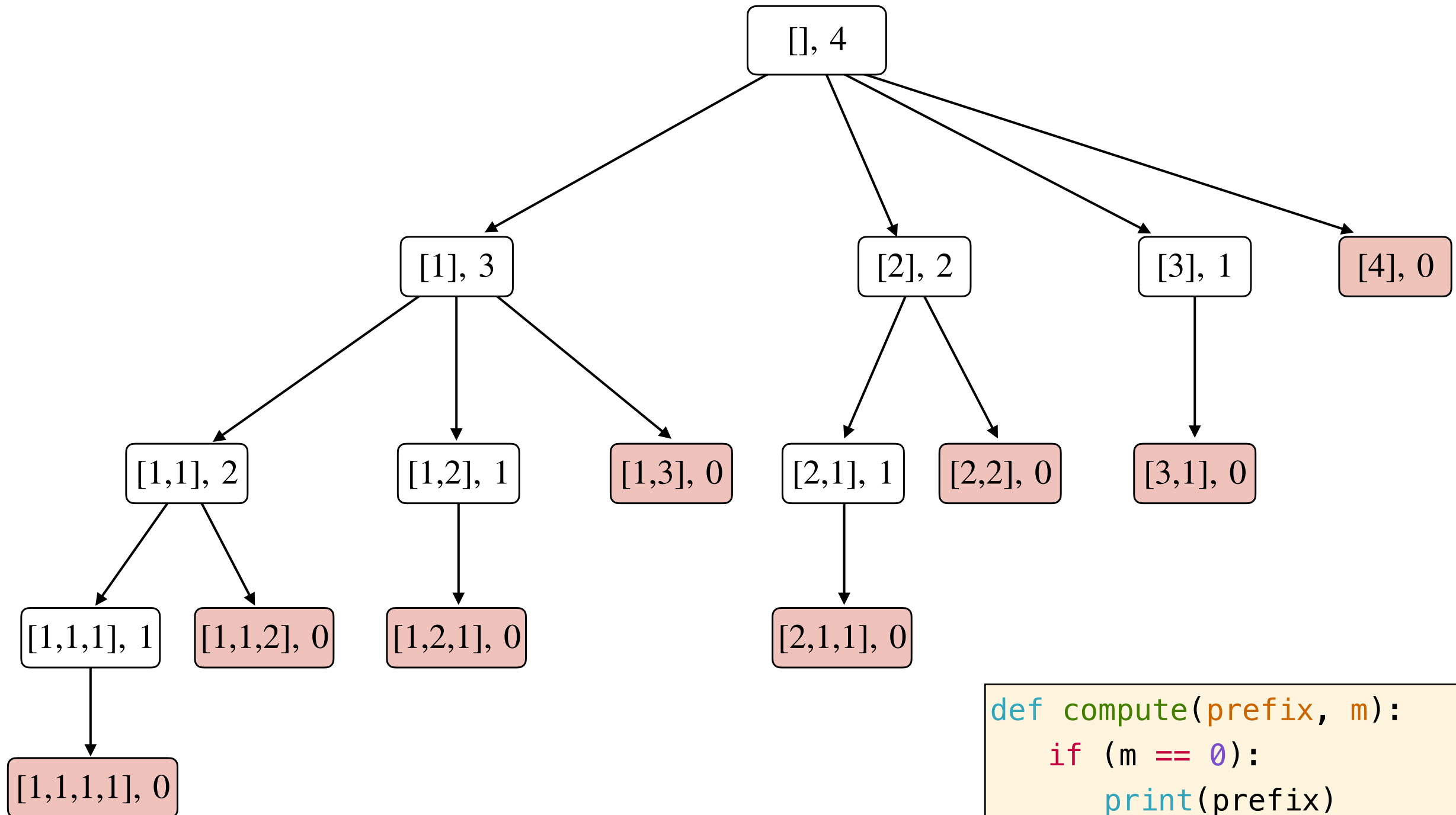
Exercise: Can you modify the code so that no new lists are generated, and all the changes are in-place?

Recursion tree

Recursion tree

```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
compute([], 4)
```


Recursion tree



```
def compute(prefix, m):  
    if (m == 0):  
        print(prefix)  
        return  
  
    for i in range(1, m+1):  
        compute(prefix+[i], m-i)  
  
compute([], 4)
```


Substrings of a string (having distinct characters)

Substrings of a string (having distinct characters)

Recursive approach:

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    # exclude s[i]  
    compute(s, prefix, i+1)  
  
    # include s[i]  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```

Substrings of a string (having distinct characters)

Recursive approach:

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    # exclude s[i]  
    compute(s, prefix, i+1)  
  
    # include s[i]  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```

Invariant:

Compute(s, prefix, i) generates all possible substrings w of s[i:] and prints each as prefix + w.

Substrings of a string (having distinct characters)

Recursive approach:

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    # exclude s[i]  
    compute(s, prefix, i+1)  
  
    # include s[i]  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```

Invariant:

Compute(s, prefix, i) generates all possible substrings w of s[i:] and prints each as prefix + w.

Base case (i = len(S)) holds.

Substrings of a string (having distinct characters)

Recursive approach:

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    # exclude s[i]  
    compute(s, prefix, i+1)  
  
    # include s[i]  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```

Invariant:

`Compute(s, prefix, i)` generates all possible substrings `w` of `s[i:]` and prints each as `prefix + w`.

Base case ($i = \text{len}(S)$) holds.

Let W_i be the set of all substrings of `s[i:]`, then the inductive step holds because:

- `compute(s, prefix, i+1)` prints all strings of form `prefix+w` where $w \in W_{i+1}$,
- `compute(s, prefix+s[i], i+1)` prints all strings of form `prefix +s[i]+ w` where $w \in W_{i+1}$.

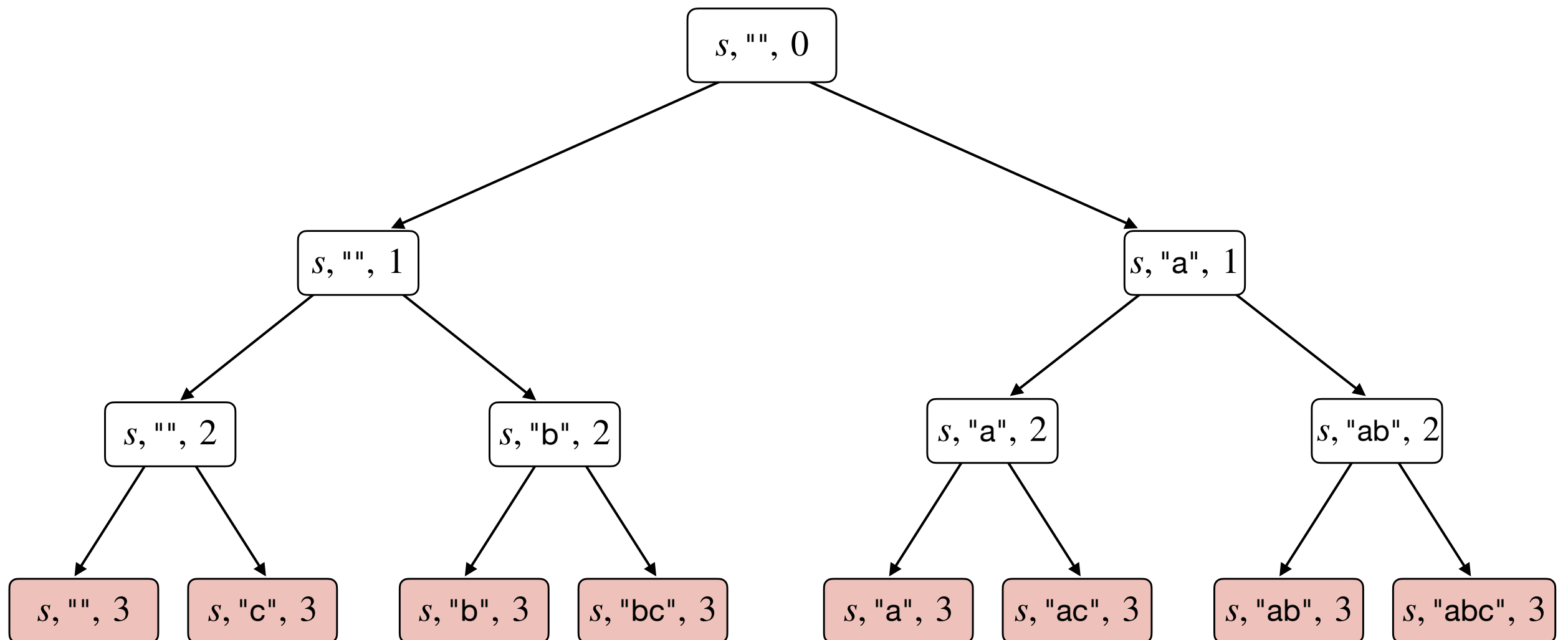
Recursion tree

Recursion tree

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    compute(s, prefix, i+1)  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```


Recursion tree

```
def compute(s, prefix, i):  
    if i == len(s):  
        print(prefix)  
        return  
  
    compute(s, prefix, i+1)  
    compute(s, prefix+s[i], i+1)  
  
s = "abc"  
compute(s, "", 0)
```



Dictionaries

Dictionary

Dictionary

Dictionaries in python are **mutable** data-structures storing **key,value** pairs, where keys are necessarily distinct.

Dictionary

Dictionaries in python are **mutable** data-structures storing **key,value** pairs, where keys are necessarily distinct.

Note: Dictionaries are not indexable by integers.

Dictionary

Dictionaries in python are **mutable** data-structures storing **key,value** pairs, where keys are necessarily distinct.

Note: Dictionaries are not indexable by integers.

Example:

Dictionary

Dictionaries in python are **mutable** data-structures storing **key,value** pairs, where keys are necessarily distinct.

Note: Dictionaries are not indexable by integers.

Example:

```
person = {"name": "Alice", "age": 25}

print(person["name"])      # prints: Alice

person["age"] = 50          # updates existing key
```

Dictionary

Dictionary

Program

```
D = {"Alice":80, "Bob":90, "Chen": 85}

print(D["Alice"])    # value of key

if "Alice" in D:      # key's membership
    print("Alice is present")

for key in D:         # key iteration
    print(key)

D["John"] = 90        # adding/updating

del D["Alice"]        # deleting

print(D.keys())       # all keys
print(D.values())     # all values
print(D.items())      # (key,value) pairs
```


Dictionary

Program

```
D = {"Alice":80, "Bob":90, "Chen": 85}

print(D["Alice"])    # value of key

if "Alice" in D:      # key's membership
    print("Alice is present")

for key in D:         # key iteration
    print(key)

D["John"] = 90        # adding/updating

del D["Alice"]        # deleting

print(D.keys())       # all keys
print(D.values())     # all values
print(D.items())      # (key,value) pairs
```

Output

```
80

Alice is present

Alice

Bob

Chen

dict_keys(['Bob', 'Chen', 'John'])

dict_values([90, 85, 90])

dict_items([('Bob', 90), ('Chen', 85), ('John', 90)])
```

Computing frequency using dictionaries

Computing frequency using dictionaries

Program

```
my_list = [9, 9, 1, 3, 1, 9]

frequency = {}
for item in my_list:
    if item in frequency:
        frequency[item] += 1
    else:
        frequency[item] = 1

print(frequency)
```

Computing frequency using dictionaries

Program

```
my_list = [9, 9, 1, 3, 1, 9]

frequency = {}
for item in my_list:
    if item in frequency:
        frequency[item] += 1
    else:
        frequency[item] = 1

print(frequency)
```

Output

```
{9: 3, 1: 2, 3: 1}
```

Product listing / average score using Dictionaries

Product listing / average score using Dictionaries

Program

```
product = {  
    "name": "Headphones",  
    "price": 4999,  
    "reviews": [  
        {"user": "Alex", "score": 5},  
        {"user": "Bob", "score": 3},  
        {"user": "Chen", "score": 4}  
    ]  
}  
  
total = 0  
  
for dict in product["reviews"]:  
    total = total + dict["score"]  
  
print(total/len(product["reviews"]))
```

Product listing / average score using Dictionaries

Program

```
product = {  
    "name": "Headphones",  
    "price": 4999,  
    "reviews": [  
        {"user": "Alex", "score": 5},  
        {"user": "Bob", "score": 3},  
        {"user": "Chen", "score": 4}  
    ]  
}  
  
total = 0  
  
for dict in product["reviews"]:  
    total = total + dict["score"]  
  
print(total/len(product["reviews"]))
```

Output

4.0